Lounastutka

Lounastutka is a containerized full-stack app with:

- Frontend: React Router app
- Backend: Bun HTTP service
- Database: PostgreSQL

The repository includes two deployment modes:

- Local development with Docker Compose
- Production-style Docker Swarm stack with Traefik and monitoring services

Services Overview

Development (`compose.dev.yaml`):

- Frontend: `http://localhost:5173`
- Backend: `http://localhost:3001`
- PostgreSQL: `localhost:5432`

Swarm (`compose.yaml`):

- Traefik edge router: `http://localhost`
- Traefik dashboard: `http://localhost:8080`
- Grafana: `http://localhost:3000`
- Prometheus and Loki are internal by default

Documentation Map

- Installation and run instructions: Installation
- Frontend module notes: Frontend
- Backend module notes: Backend

Build Documentation

From repository root:

```
mkdocs build
```

Serve locally with live reload:

```
mkdocs serve
```

Installation

This guide reflects the current setup of the repository.

Prerequisites

- Docker Engine
- Docker Compose plugin (`docker compose`)
- Optional for docs: Python and MkDocs

Quick Start (Local Development)

Run from repository root:

```
docker compose -f compose.dev.yaml up
```

This starts:

- Frontend on `http://localhost:5173`
- Backend on `http://localhost:3001`
- PostgreSQL on `localhost:5432`

Stop services:

```
docker compose -f compose.dev.yaml down
```

Stop and remove volumes:

```
docker compose -f compose.dev.yaml down -v
```

Production-Style Swarm Deployment

Initialize Swarm (once):

```
docker swarm init
```

Build images:

```
docker build -t lounastutka-backend:latest ./Backend
docker build -t lounastutka-frontend:latest ./Frontend
```

Deploy stack:

```
docker stack deploy -c compose.yaml lounastutka
```

Useful checks:

```
docker stack services lounastutka
docker service ps lounastutka_app
docker service ps lounastutka_frontend
```

Swarm endpoints:

- App (via Traefik): `http://localhost`
- Traefik dashboard: `http://localhost:8080`
- Grafana: `http://localhost:3000`

Remove stack:

```
docker stack rm lounastutka
```

Notes

- In local development, frontend and backend source folders are bind-mounted into containers.
- Prometheus and Loki are included in Swarm mode and are internal by default.
- For real production, move credentials from Compose files into a secret manager.

Backend

Basic information regarding development and structure of the backend is described here. This is not supposed to be an indepth manual and explanation of the design decisions but a high overview of the initial idea for the structure and whats where.

Setup:

You need to have bun.js installed: bun.js installation

Install dependencies for the application backend:

```
[project_root]$ cd Backend/
[Backend]$ bun install
```

To run the server as standalone instance instead of docker container:

```
[Backend]$ bun run index.ts
```

For better testing it is recommended to run the whole setup with:

```
# To build and start the services:
[project_root]$ docker compose -f compose.dev.yaml up --build

# To stop them:
[project_root]$ docker compose -f compose.dev.yaml down

# And to remove the volumes as well:
[project_root]$ docker compose -f compose.dev.yaml down -v
```

Maintenance & Development

The backend is divided to the following structure:

```
[Backend]$ tree -I node_modules -I docs
.
├── bun.lock
├── controllers
│   ├── auth.controller.ts
│   ├── protected.controller.ts
│   ├── restaurant.controller.ts
│   └── types.ts
├── database
│   ├── helpers.ts
│   └── models.ts
├── Dockerfile
├── index.ts
├── middleware
│   ├── auth.middleware.ts
│   ├── auth.validator.ts
│   ├── error.middleware.ts
│   └── validator.ts
├── package.json
├── README.md
├── routes
│   ├── auth.routes.ts
│   ├── protected.routes.ts
│   └── restaurant.routes.ts
├── services
│   ├── auth.service.ts
│   ├── protected.service.ts
│   └── restaurant.service.ts
├── tsconfig.json
└── utils
    └── error.ts
```

Current dependencies can be found in the `package.json` file and package versions should be updated when vulneraibility patches come.

About the files, folders and their ideas:

index.ts: Entry to the backend application, defines the used features, e.g. express, cors etc.

utils/: Global utility methods and classes should lay here as they can be easily imported to the relevant source code files. E.g AppError.

routes/: routes that can be called to the backend defined here and which middleware are used, e.g. with this defined in index.ts: `app.use("/api/protected", protectedRoutes);` and in routes/protected.routes.ts `router.get("/hello-there", authenticateToken, helloThere);`.

controllers/: Here are defined the methods that handle calling appropriate services and acts as HTTP endpoint, the controllers deal with the request and response objects and if something goes wrong, calls the next middleware for errors or responds back to the caller with the

response object.

services/: These are called by the controllers and these methods handle the business logic of the backend application, calling queries to the database, potentially other services, e.g. the microservice etc.

database/: Acts as API to the database service, containing the database schema based typescript models and including the methods that make the database SQL queries via bun.js SQL library. It also has the connection initialization and its usage should be as easy as importing it to a service file and using e.g. `await db.findUserByID(...)`.

middleware/: This folder includes all of the custom middleware methods such as email & password verification and token verification and custom error handler for express.

API Documentation

The API documentation is built from the developer written `typedoc` and `JSDoc` style code comments and typedoc parsed method declarations. This is not the best way to do API documentation but considering the small codebase at this point it seemed to suffice, as the project grows it is beneficial to move towards e.g. OpenAPI style documentation format. The documentation index.html can be created with:

```
[Backend]$ bun run typedoc --entryPointStrategy Expand controllers/* middleware/*
```

, this should create the new html files under `Backend/docs/` folder.

NOTE: For better documentation, strong typing is recommended as typedoc can and will complain if there are 'any' types around.

Error handling

Currently there exists only the generic error handling code that shows the end-user generic error code rather than showing the full stack from backend as it could potentially contain protected information and in the end wouldn't help a normal user. This relies on the fact that when developing and using error messages, it is important to differentiate what types of errors the user should see and what should remain under the maintainers logs, e.g. 4xx codes vs 5xx codes.

NOTE: This project was created using `bun init` in bun v1.3.10. Bun is a fast all-in-one JavaScript runtime.

Microservice (Scraper Service)

This microservice is responsible for retrieving restaurant and lunch menu data for the Lounastutka application.

It acts as a dedicated data provider between the backend and external (or simulated) data sources.

Responsibilities

- Accept a restaurant URL from the backend
- Return structured restaurant data

Architecture Role

```
Frontend → Backend → Microservice → Backend → Database → Frontend
```

The microservice is **not exposed publicly** and is only accessible within the Docker network.

Overview

The service exposes a simple HTTP API that:

- Receives a restaurant URL
- Returns structured restaurant + menu data
- Provides either real or mock data depending on the source

Behavior

- **Aalef URL (`https://www.aalef.fi/#ravintolat`)**
- Attempts to scrape real menu data for the current day
- **All other URLs**

- Returns randomized mock data (for now)
- Locations are constrained within the Lappeenranta area

API

`POST /scrape`

Request:

```
{
  "url": "https://example.com"
}
```

Example response:

```
{
  "id": 1,
  "type": "restaurant",
  "position": [61.05, 28.19],
  "name": "Restaurant Name",
  "category": "Ravintola",
  "stars": 4.5,
  "reviews": 120,
  "address": "Example street 1",
  "description": "Description",
  "todayHours": "10:00-15:00",
  "lunchTime": "11:00-14:00",
  "priceLevel": "Lunch 12 EUR",
  "phone": "000000",
  "website": "https://example.com",
  "tags": ["lunch"],
  "todayMenu": ["Food 1", "Food 2"]
}
```

`GET /health`

Returns service health status:

```
{ "status": "ok" }
```

Running locally

```
docker compose up --build microservice
```

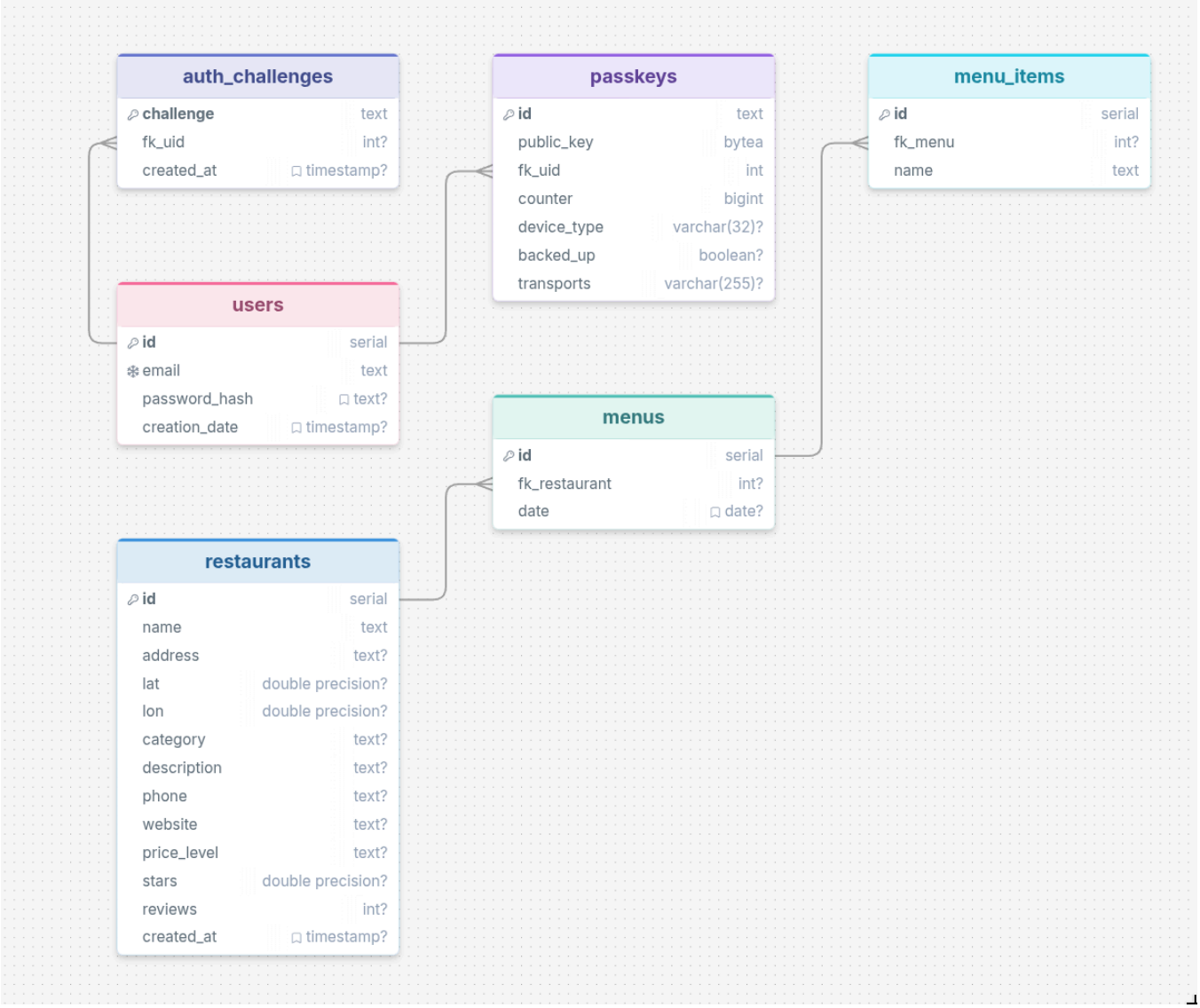
PostgreSQL DB

The project uses PostgresSQL relational database for storing relevant user and restaurant information. The database is run as a separate docker service and accessed by the backend docker service via the defined helpers in `Backend/database/`.

The .sql files under `Database/init/` are mounted to the docker service environment and executed during the first initialization of the database.

The user `lounastutka` is authorized to access the database and the environment variables should be set correctly for that user to have access from the backend. This setup is different for production and for local development and is described in `Environment` -section. Examples can be seen in .env-example and within the docker compose files.

Diagram built automatically with https://drawsql.app/ by importing the database schema.sql.


auth_challenges
challenge text
fk_uid int?
created_at timestamp?
passkeys
id text
public_key bytea
fk_uid int
counter bigint
device_type varchar(32)?
backed_up boolean?
transports varchar(255)?
menu_items
id serial
fk_menu int?
name text
users
id serial
email text
password_hash text?
creation_date timestamp?
menus
id serial
fk_restaurant int?
date date?
restaurants
id serial
name text
address text?
lat double precision?
lon double precision?
category text?
description text?
phone text?
website text?
price_level text?
stars double precision?
reviews int?
created_at timestamp?

Frontend Documentation Landing Page

The frontend is a Vite + React + TanStack Router single-page app for browsing lunch places on a map and in a list view, with authentication and restaurant management flows layered on top.

What This Frontend Does

- Shows lunch places on an interactive Leaflet map.
- Provides a searchable and filterable restaurant list sorted by distance.
- Uses user location to compute and display distance labels.
- Supports password and passkey based authentication flows.
- Lets authenticated users preview and submit restaurant data.
- Exposes a small component playground for shared UI pieces.

Tech Stack

- React + TypeScript
- Vite
- TanStack Router with file-based routing
- Tailwind CSS
- Leaflet and React Leaflet
- Framer Motion
- WebAuthn support via @simplewebauthn/browser

Core Areas

- Map home view: interactive map, visible restaurant sidebar, selection sheet, and location controls.
- List view: search, type filters, distance sorting, and expandable restaurant cards.
- Settings view: login, registration, and authenticated restaurant submission tools.
- Shared UI: map pins, restaurant cards, search bar, sidebar, bottom navigation, and modal/sheet components.
- Data and services: restaurant fetchers, auth token helpers, and API wrappers.

Project Structure

```
Frontend/
    app/
        components/        # Shared UI, map behavior, and restaurant UI
        data/              # Local demo and typed place data
        hooks/             # Shared hooks such as usePlaces
        routes/            # Route components for /, /list, /settings, /components
        services/          # API helpers for public and protected endpoints
        utils/             # Utility functions such as distance formatting
        app.css            # Global styles
        main.tsx           # App bootstrap and router provider
        routeTree.gen.ts   # Generated TanStack Router tree
    docker/nginx/        # Nginx config used by the production container
    public/              # Static assets
    Dockerfile           # Frontend production image
```

Routes

- `/` - Map-first home view
- `/list` - Searchable and filterable restaurant list
- `/settings` - Authentication and restaurant submission view
- `/components` - Component playground and visual demo page

Data Flow And API Notes

- Public restaurant data is fetched through frontend helpers that call backend routes under `/api`.
- Authenticated requests attach Authorization: Bearer token when a token exists in localStorage.
- The JWT token is stored under the key jwt, and a 401 response clears it automatically.
- Frontend development works best when the frontend and backend are run together or when `/api` is proxied to the backend.

Local Development

From Frontend/:

```
npm install
npm run dev
```

Default dev URL:

- http://localhost:5173

Available scripts:

- `npm run dev` - Start the Vite dev server
- `npm run build` - Create the production build in build/client
- `npm run typecheck` - Run TypeScript checks

Recommended Full-Stack Local Run

From the repository root:

```
docker compose -f compose.dev.yaml up
```

This is the easiest way to run the frontend, backend, and database together with the expected routing.

Production Build Output

- Vite output directory: build/client
- Served by Nginx in containerized deployments

Docker Image

Build from the repository root:

```
docker build -t lounastutka-frontend:latest ./Frontend
```

The production image:

- Builds static assets with Node
- Serves assets with Nginx
- Uses SPA fallback routing
- Proxies /api and /auth to the backend container

Map and List View Behavior

Map View (`/`)

The map view is the primary interface for browsing restaurants. It combines an interactive Leaflet map with sidebar and mobile sheet overlays for restaurant discovery and selection.

Core Components

- **MapContainer**: Root React Leaflet container rendering OpenStreetMap tiles (OSM).
- **MapBehavior components**: Pluggable controllers handling map interactions:
- `SetViewOnClick`: Listener for map clicks; clears restaurant selection when user taps the map.
- `MapBoundsTracker`: Tracks visible restaurants within current map bounds; updates sidebar and sheet in real time as user pans or zooms.
- `MapSelectionFocus`: When a restaurant is selected, recenters the map to keep the pin visible while accounting for mobile UI overlays (sheet + bottom nav).
- `UserLocationControl`: Floating button to locate user; displays a circle marker at user position; handles geolocation errors.

Interaction Flow

1. **Load**: Map initializes at hardcoded center (Lappeenranta); fetches all restaurants from API.
2. **Pan/Zoom**: Map bounds change → `MapBoundsTracker` filters visible restaurants → sidebar updates.
3. **Select**: User clicks pin or sidebar item → `setRestaurantSelected()` → `MapSelectionFocus` animates map to keep selection in frame.
4. **Deselect**: User clicks map (not a pin) or closes sheet → selection clears.
5. **Search**: Global search bar (top-center) filters visible restaurants by name, category, or tags.

Responsive Behavior

- **Desktop**: Sidebar is persistent on the left; shows full list of visible restaurants.
- **Mobile**: Sidebar is not visible; selected restaurant shows in a container under search bar so that map is still visible a bit
- **Recentering**: On mobile, `MapSelectionFocus` calculates optimal center point to avoid obscuring pins behind the sheet and navigation.

List View (`/list`)

The list view displays all restaurants in a vertically scrollable feed, with client-side search and filtering.

Core Features

- **Search**: Text input filters restaurants by name, category, or tags (case-insensitive substring match).
- **Type Filter**: Pills for "Kaikki" (all), "Ravintola" (restaurant), "Pizza", and "Kasvis" (vegan).
- **Distance Sort**: Sorted by distance from user location if available; falls back to no distance if geolocation fails.
- **Expandable Cards**: Each restaurant card expands on click to show full details (description, hours, menu, contact).
- **Animations**: Framer Motion animates card entrance and expansion smoothly.

Data Flow

1. Fetch all restaurants from API via `usePlaces()`.
2. Get user location via `useUserLocation()`.
3. Compute distance for each restaurant; filter by search query and active type.
4. Sort by distance; memoize result to avoid recomputation on every render.
5. Render filtered list with animations.

State Management

- `search`: Current search query.
- `activeType`: Selected restaurant type filter (or null for all).
- `expandedId`: ID of currently expanded card (or null).
- Memoization via `useMemo` ensures filtering and sorting only run when inputs change.

Key Behaviors

Distance Calculation

Both views use `calculateDistanceMeters(from, to)` to compute geodetic distance from user to restaurant. The distance is displayed as a human-readable label (e.g., "2.5 km") via `formatDistance()`.

Location Permissions

- If user denies geolocation, distance fields remain empty; sorting falls back to default order.
- `UserLocationControl` (map only) shows error message if geolocation fails (e.g., timeout).

Performance

- `useMemo` caches filtered/sorted results to avoid recomputation when only UI state changes.
- Map bounds tracking is event-driven (moveend/zoomend) to avoid constant filtering.
- Restaurant card expansion is local state; no re-renders of sibling cards.

Component Catalog and Design Rules

Shared UI Components

Button

Location: `app/components/Button.tsx`

Animated, customizable button with loading state support.

- **Variants**: `primary` (blue, shadow), `secondary` (gray), `outline` (transparent border)
- **Props**: `isLoading` (shows spinner), `variant`, standard HTML button attributes
- **Behavior**: Scale animation on hover (1.02x) and tap (0.98x); disabled when loading or via `disabled` prop
- **Usage**: Form submissions, navigation, toggle actions

SearchBar

Location: `app/components/SearchBar.tsx`

Search input with search icon and blue background.

- **Props**: `value`, `onChange`, `placeholder`, `className`
- **Behavior**: Max length 100 chars; icon on left; neutral text color; responsive width
- **Usage**: Filter restaurants by name/category/tags in map or list view

BottomNav

Location: `app/components/BottomNav.tsx`

Fixed bottom navigation with three routes: Map, List, Settings.

- **Behavior**: Active tab has blue background; uses react-router for navigation
- **Design**: Rounded bar, icon + label, responsive (stack on mobile)
- **Usage**: Primary navigation on mobile views

Rating

Location: `app/components/Rating.tsx`

Display star rating and review count.

- **Display**: Star icon, number of stars, review count in parentheses
- **Behavior**: Clickable link (currently routes to `/`)
- **Usage**: RestaurantCard header to show restaurant rating

MapPin

Location: `app/components/MapPin.tsx`

Map markers with restaurant type icons.

- **MapPinVisual**: Visual representation (icon + blue circle)
- **MapPinMarker**: Leaflet marker with icon caching and popup
- **Types**: `restaurant`, `pizza`, `vegan` (different icons)
- **Design**: Configurable size; icon-centric; cached for performance
- **Usage**: Leaflet map layer for each restaurant

SideBar

Location: `app/components/SideBar.tsx`

Desktop-only sidebar (md: breakpoint) showing visible restaurants.

- **Behavior**: Toggleable (collapse/expand); shows restaurant cards; highlights selected
- **Design**: Gradient background; toggle button; spring animation for cards
- **Usage**: Desktop map view; shows filtered list of restaurants in view bounds

MobileRestaurantSheet

Location: `app/components/MobileRestaurantSheet.tsx`

Mobile-only bottom sheet showing selected restaurant details.

- **Behavior**: Slides in from bottom; shows full restaurant info; dismissible
- **Design**: Mobile-optimized; overlays bottom nav
- **Usage**: Mobile map/list views; displays restaurant details on selection

RestaurantCard

Location: `app/components/RestaurantCard.tsx`

Expandable card displaying restaurant summary and details.

- **Collapsed**: Header (name, category, rating, distance)
- **Expanded**: Meta grid (hours, price, description), menu items, contact section
- **Animation**: Framer Motion smooth height/opacity transition on expand
- **Composition**: CardHeader, CardMetaGrid, CardMenu, CardContactSection subcomponents
- **Usage**: List view, sidebar, mobile sheet; reusable across contexts

RestaurantCard Subcomponents

Location: `app/components/_RestaurantCard/`

Modular card sections:

- **CardHeader**: Name, category, rating, distance
- **CardMetaGrid**: Hours, price, description in a 2-column grid
- **CardMenu**: Today's menu items as a list
- **CardContactSection**: Phone, website, address, tags
- **CardActions**: "Show More Info" button with expand/collapse state

FavouriteButton

Location: `app/components/FavouriteButton.tsx`

Heart icon button to mark/unmark favorites (if implemented).

- **Design**: Icon-based; heart outline / filled
- **Usage**: Card actions or detail views

AuthModal

Location: `app/components/AuthModal.tsx`

Login and registration interface with password and passkey flows.

- **Behavior**: Tab-based (login/register); passkey support via WebAuthn
- **Design**: Form fields, buttons, error handling

- **Usage**: Settings page for user authentication

UserLocationProvider

Location: `app/components/UserLocationProvider.tsx`

Context provider managing geolocation state and location-based features.

- **Exports**: `useUserLocation()` hook
- **State**: `position`, `isLocating`, `errorMessage`, `locateUser()`
- **Usage**: Global location tracking across map and list views

Design Patterns

Tailwind Styling

All components use Tailwind CSS with:

- **Colors**: Primary (blue), neutral (backgrounds), dark (text)
- **Spacing**: Consistent padding/gaps (4px, 8px, 12px, 16px increments)
- **Borders/Radius**: `rounded-2xl` (cards), `rounded-4xl` (buttons/search), `rounded-3xl` (icon backgrounds)
- **Shadows**: `shadow-lg`, `shadow-md`, `shadow-sm` for depth; primary color shadows for accent
- **Responsive**: `md:` breakpoint for desktop layout; mobile-first default

Animations

- **Framer Motion**: Spring animations for layout shifts and card expansions; instant transitions for state changes
- **Button interactions**: `whileHover` (scale 1.02), `whileTap` (scale 0.98)
- **Card expansion**: Height + opacity animation; 240ms duration
- **Entrance animations**: List items fade in with staggered delay

Composition Strategy

- **Atomic**: Buttons, search bars are self-contained utilities
- **Modular subcomponents**: RestaurantCard delegates header/sections to subcomponents for readability
- **Context providers**: Location tracking via context rather than prop drilling
- **Route-aware**: BottomNav and AuthModal adapt to current route

Accessibility Considerations

- **Semantic HTML**: `<button>`, `<input>`, proper form structure
- **ARIA labels**: Location button has `aria-label`
- **Keyboard support**: Inputs auto-focus when visible; tab navigation through buttons
- **Color contrast**: Blue primary + white/dark text meets WCAG AA standards

Type Safety

All components accept TypeScript interfaces for props:

```
export interface ButtonProps extends HTMLMotionProps<"button"> {
  isLoading?: boolean;
  variant?: "primary" | "secondary" | "outline";
}
```

Performance

- **Icon caching**: `MapPin` caches rendered Leaflet icons by type/size
- **Memoization**: Card list renders memoized; expensive filters run only when inputs change
- **Event delegation**: Map events (moveend, zoomend) trigger bounds updates, not every pan frame

Architecture

The description of the architecture of the system.

The diagrams are built using PlantUML with some diagrams utilizing the C4 model extension available here: https://github.com/plantuml-stdlib/C4-PlantUML#deployment-diagram

If and when changes are made to the diagrams (and when the architecture changes), they can be recreated under `docs/architecture/imgs` with:

```
[project_root]$ cd docs/architecture/
[architecture]$ plantuml diag_src/*.puml -o ../imgs
```

NOTE You need to have 'plantuml' installed, the C4 model extension should be fetched automatically from the github repository.

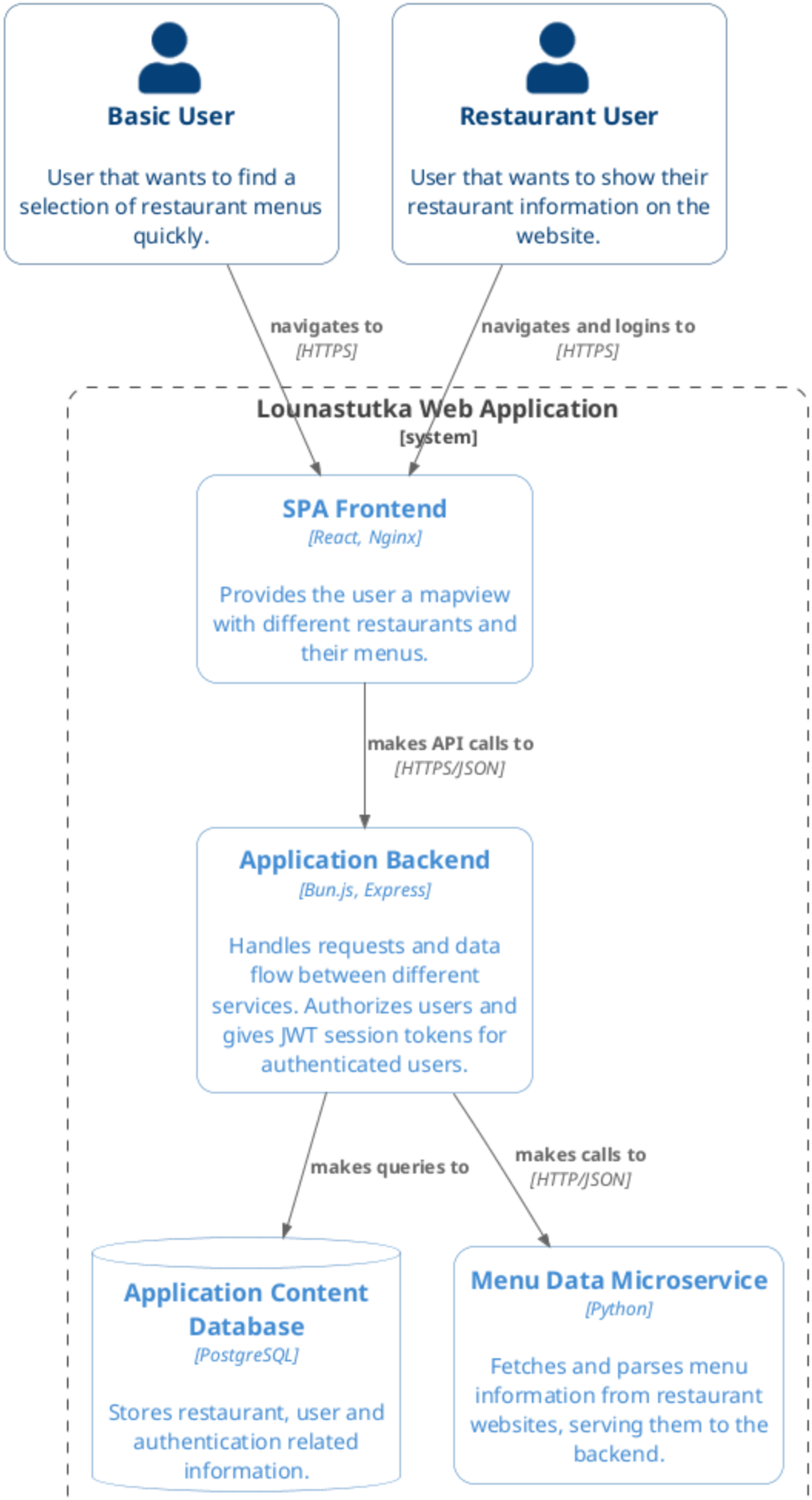
The Architecture

The main application components are described here, without the infrastructure components such as traefik, grafana, portainer, docker etc. as they are shown within the deployment diagram. The diagram style utilized here is the C4 models "Container" view and is built with `plantuml`.

Containers:

- **Basic User:** User that does not necessarly want to login but wants to check local restaurant menus via interactive map.
- **Restaurant User:** User who might want to add their restaurant information on the website for other users to see.
- **Frontend (bunjs nginx react):** Responsible for showing user the main content of the application. This is the service user directly interacts with, others are internal and only accessible through APIs.
- **Backend (bunjs express):** Handles the data flow between frontend, microservice and database.
- **Microservice (python):** Scrapes a website based on the given URL and tries to gather menu data.
- **Database (postgresql):** Stores applications content such as information about the restaurants as well as metadata about the users that want to authenticate.

Main Component Architecture - Lounastutka System

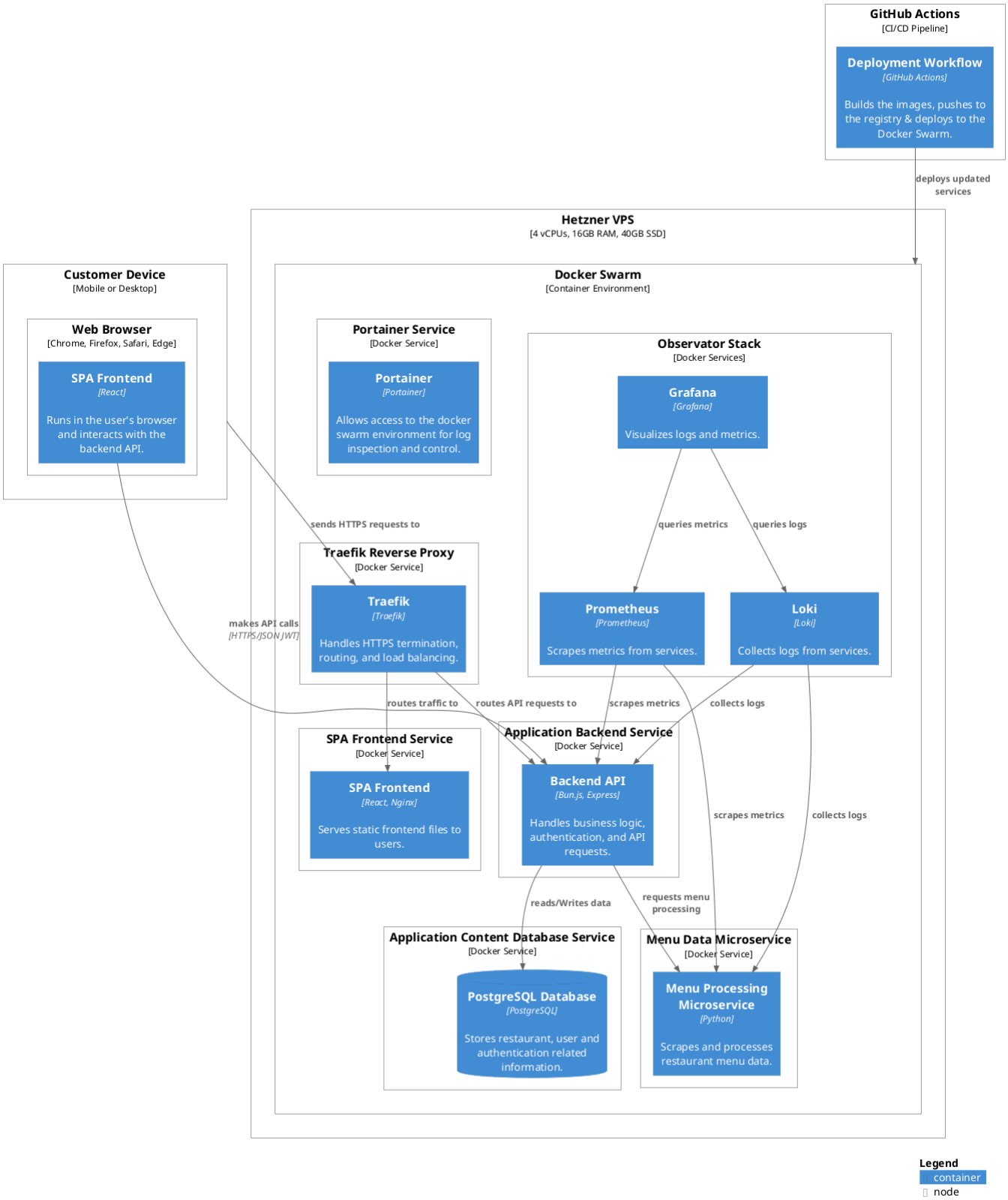


Legend
person
container
system boundary

Deployment diagram

The system with infrastructure services such as Traefik, Grafana, Loki and prometheus frameworks. The internal services are all deployed on a single VPS at Hetzner server farm. The github actions is external system and connects to the internal system via CI/CD pipeline.

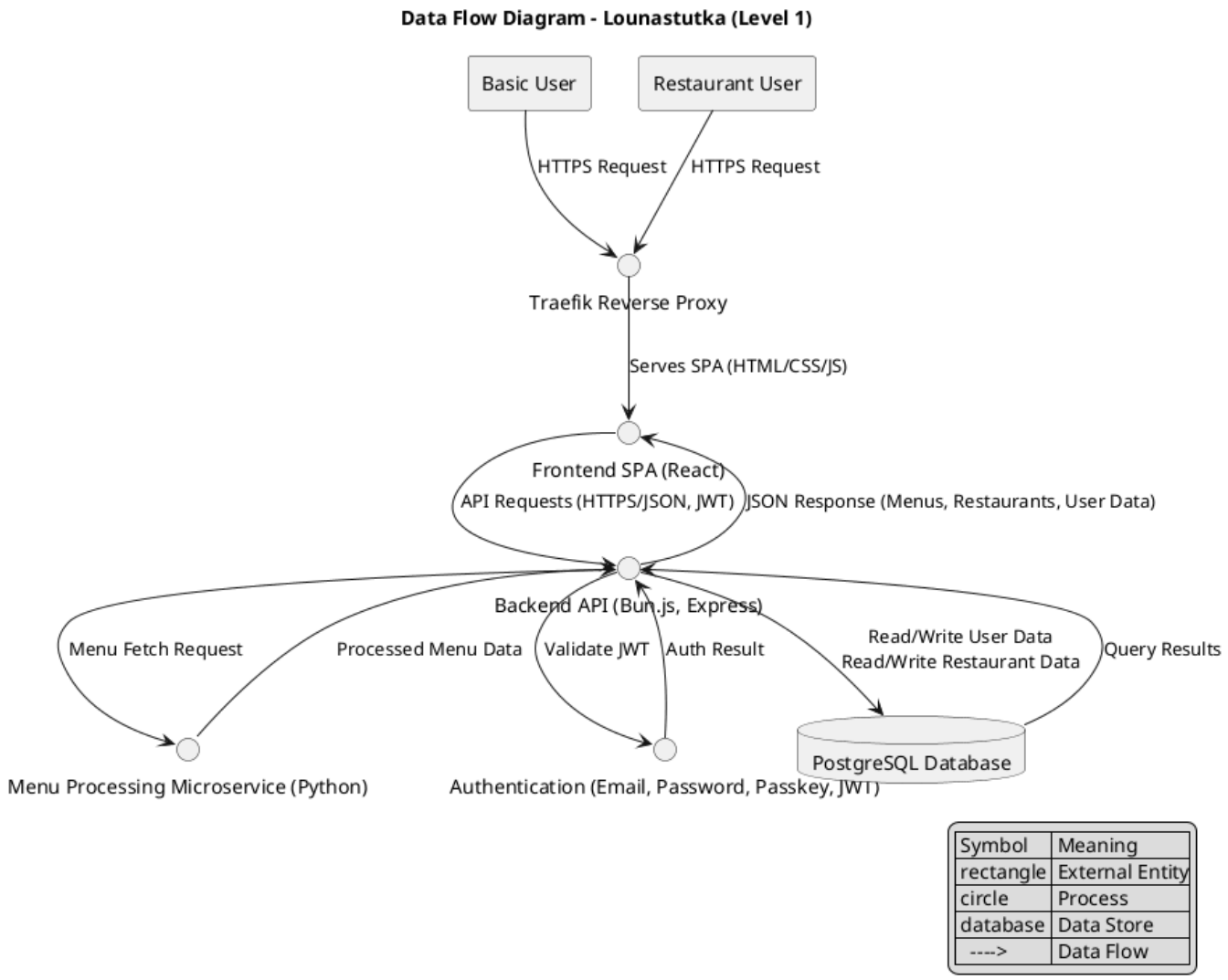
Deployment Diagram - Lounastutka System



Data flow diagram

The diagram shows the Data Flow Diagram Level 1 view with the "Yourdon and Coad"-style as descriped in lucidcharts website here: https://www.lucidchart.com/pages/data-flow-diagram

Most of the data flows are bi-directional, and the users interaction with traefik are described only as one-directional due to the initial navigation to the website (SPA frontend) from which the data flows between the software system. Circles are the processes between which data flows in the manner as the arrows describe. Rectangles are the external entities such as users, the github actions could in theory be described here as it is external entity from which data is flowing to the system when updates happen but it is shown in the deployment diagram and felt out of place here. The database is the only data store element within the system.

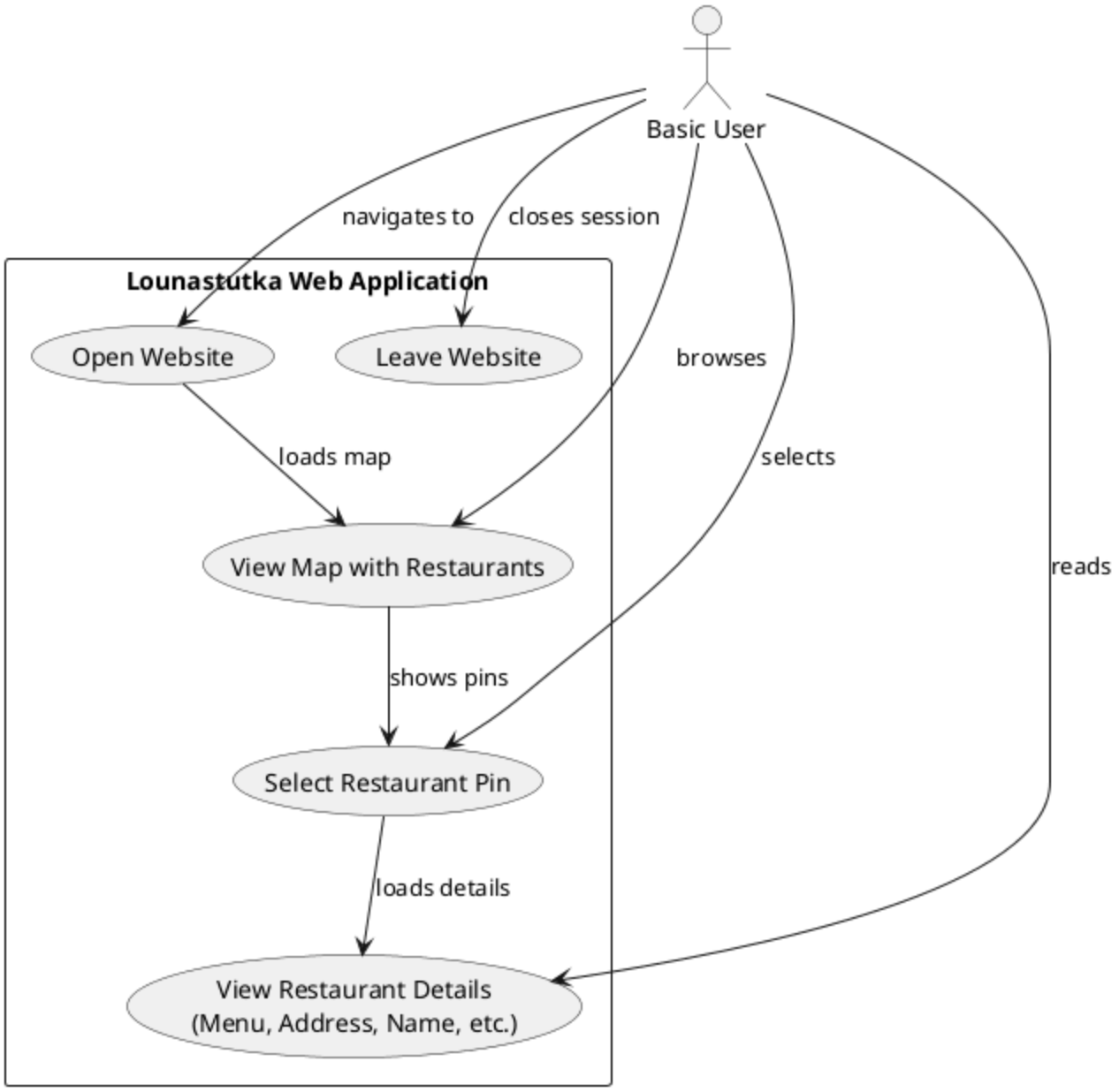


Use-case diagrams

The application has two distinct use-cases, the basic user use-case where a user wants to just see whats on the menu today on local restaurants and then close the site.

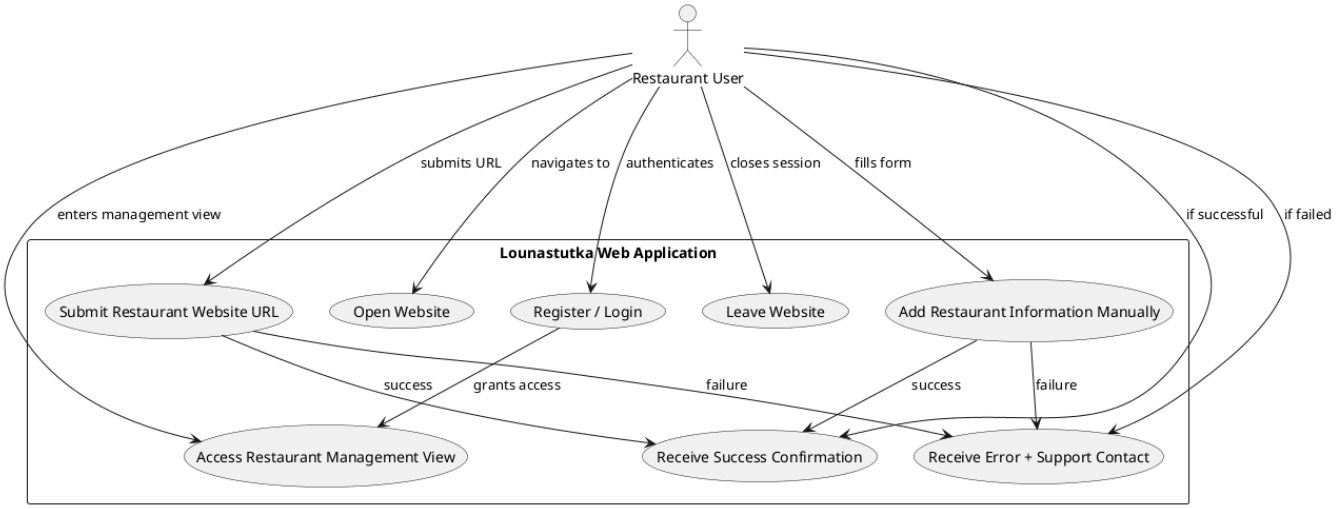
The other more complex use-case is for the restaurant users who might want to add their restaurant information to the website to be seen by the normal users, by either manually typing the information or providing an URL that is used to fetch the information from the restaurants website. As the addition might fail due to errors in data transfer, the user is given some additional information about the status.

Use Case - Basic User Browsing Restaurant Menus



Symbol	Meaning
Rectangle	Boundary
Stickman	User
Ellipse	Use-case
Arrow	Action

Use Case - Restaurant User Adding Restaurant Information



Symbol	Meaning
Rectangle	Boundary
Stickman	User
Ellipse	Use-case
Arrow	Action

Hetzner

Lounastutka is deployed on a Hetzner Cloud VPS. Hetzner was chosen because it was one of the cheapest suitable hosting options for the project, while still providing European infrastructure, simple VPS management, and DNS automation support.

The production environment runs on a single Hetzner Cloud server using the `CPX32` server type.

Why Hetzner

Hetzner was selected for these reasons:

- it was one of the lowest-cost options that still provided enough resources for the application
- the servers are hosted in Europe
- Hetzner Cloud has a straightforward VPS model
- Hetzner DNS can be controlled through an API
- Traefik supports Hetzner DNS for Let's Encrypt DNS challenges
- the setup works well with Docker Swarm and GitHub Actions deployments

The project uses an external domain name, but DNS for that domain is controlled through Hetzner DNS.

Server Type

The production VPS uses:

Setting	Value
Provider	Hetzner Cloud
Server type	`CPX32`
CPU platform	AMD
vCPUs	4
Memory	8 GB
Disk	160 GB

Setting	Value
Maximum monthly price	EUR 18.18 / month
Hourly price	EUR 0.0291 / hour
Role	Single-node production Docker Swarm host
Public networking	Paid IPv4 address
DNS provider	Hetzner DNS

The CPX32 size gives the deployment enough capacity to run the application services, PostgreSQL, Traefik, monitoring, logging, and Portainer on one VPS while keeping hosting costs predictable.

The server hosts the full production stack:

- Traefik
- frontend
- backend API
- scraper microservice
- PostgreSQL
- Prometheus
- Loki
- Promtail
- Grafana
- Portainer

The stack currently runs as a single-node deployment. Docker Swarm is still used because it gives a production-style deployment model, service labels, rolling updates, and compatibility with Traefik's Swarm provider.

IPv4 Address

The project uses a purchased IPv4 address for the production server.

This IPv4 address is used for:

- public web traffic to `lounastutka.fi`
- SSH access from GitHub Actions
- DNS `A` records
- Let's Encrypt certificate validation through Traefik and Hetzner DNS

The actual server IP should not be committed to the repository. It is stored as a GitHub Actions secret:

```
HETZNER_HOST
```

Domain And DNS

The application uses an external domain name:

```
lounastutka.fi
```

The domain itself does not need to be registered at Hetzner, but its DNS is controlled through Hetzner DNS.

This allows Traefik to request Let's Encrypt certificates using the Hetzner DNS API. The Traefik production configuration uses:

```
--certificatesresolvers.le.acme.dnschallenge=true
--certificatesresolvers.le.acme.dnschallenge.provider=hetzner
```

The Hetzner DNS API token is passed to Traefik through:

```
HETZNER_API_TOKEN
```

In GitHub Actions this value comes from the secret:

```
HETZNER_API_TOKEN
```

DNS Records

The DNS records should point to the server's purchased IPv4 address.

Production routes currently include:

Hostname	Purpose
`lounastutka.fi`	Main application frontend and API path routing.
`dashboard.swarm.lounastutka.fi`	Traefik dashboard.

Hostname	Purpose
`whoami.swarm.lounastutka.fi`	Traefik test service.
`prometheus.swarm.lounastutka.fi`	Prometheus.
`grafana.lounastutka.fi`	Grafana.
`portainer.lounastutka.fi`	Portainer.

Each hostname should resolve to the Hetzner server's IPv4 address.

The main application uses path-based routing:

URL	Target
`https://lounastutka.fi`	Frontend
`https://lounastutka.fi/api`	Backend API
`https://lounastutka.fi/scrape`	Scraper microservice

Server Directory

Deployment files are copied to:

```
/opt/lounastutka
```

GitHub Actions copies these paths to the server:

```
compose.yaml
docker/
Database/
```

Traefik certificate state is stored under:

```
/opt/lounastutka/letsencrypt/acme.json
```

The deployment workflow creates this file before running Docker Swarm deploy:

```
mkdir -p /opt/lounastutka/letsencrypt
touch /opt/lounastutka/letsencrypt/acme.json
chmod 600 /opt/lounastutka/letsencrypt/acme.json
```

The `600` permission is required because Traefik refuses to use an ACME storage file that is too open.

Docker Swarm

The Hetzner server runs Docker Swarm.

Initialize Swarm once on the server:

```
docker swarm init
```

The stack is deployed as:

```
lounastutka
```

GitHub Actions deploys the stack with:

```
docker stack deploy -c compose.yaml lounastutka --with-registry-auth
```

The `--with-registry-auth` flag is important because production images are pulled from GitHub Container Registry.

Deployment User

GitHub Actions connects to the Hetzner server over SSH.

The deployment user is configured through these GitHub Actions secrets:

Secret	Purpose
`HETZNER_HOST`	Server IPv4 address or hostname.
`HETZNER_USER`	SSH user used by the deployment workflow.
`HETZNER_SSH_KEY`	Private SSH key for the deployment user.

The deployment user must be able to:

- access `/opt/lounastutka`
- run Docker commands
- deploy Docker Swarm stacks

- read service logs for troubleshooting

Useful commands for checking server access:

```
docker stack services lounastutka
docker service ps lounastutka_traefik
docker service logs -f lounastutka_traefik
```

Firewall And Open Ports

The production server needs these public ports:

Port	Purpose
`22`	SSH deployment access.
`80`	HTTP entrypoint and Let's Encrypt redirects.
`443`	HTTPS application traffic.

Other service ports should normally stay internal to Docker networks and should be reached through Traefik instead of being exposed directly.

Persistent Data

Several services store data on the Hetzner server through Docker volumes:

Volume	Purpose
`pg_data`	PostgreSQL data.
`prom_data`	Prometheus data.
`grafana_data`	Grafana data.
`portainer_data`	Portainer data.

The server also stores Let's Encrypt certificate state in:

```
/opt/lounastutka/letsencrypt
```

These volumes and files are part of the server state. Removing them may delete the production database, dashboards, monitoring history, or certificate state.

GitHub Actions Integration

The deployment pipeline uses Hetzner in two ways:

1. It connects to the server using SSH.
2. It passes the Hetzner DNS API token to Traefik for certificate automation.

Required Hetzner-related secrets:

Secret	Used for
`HETZNER_HOST`	SSH target server.
`HETZNER_USER`	SSH login user.
`HETZNER_SSH_KEY`	SSH authentication.
`HETZNER_API_TOKEN`	DNS challenge for Let's Encrypt certificates.

The server IP and tokens should never be committed to the repository.

Operational Notes

- The CPX32 server is the single production host.
- The server has 4 AMD vCPUs, 8 GB memory, and 160 GB disk.
- The listed server price is max. EUR 18.18 per month or EUR 0.0291 per hour.
- The public IPv4 address is paid separately and used for DNS records.
- DNS for the external domain is managed in Hetzner DNS.
- Traefik uses Hetzner DNS to create and renew Let's Encrypt certificates.
- Docker Swarm runs on the server even though the current setup is single-node.
- Stateful data is stored on the server, so volume backups matter.

Troubleshooting

GitHub Actions Cannot SSH To The Server

Check:

- `HETZNER_HOST` points to the correct IPv4 address or hostname
- `HETZNER_USER` is correct
- `HETZNER_SSH_KEY` matches a public key in the server user's `authorized_keys`
- port `22` is open

- the server is running

Domain Does Not Resolve

Check the Hetzner DNS zone:

- the hostname exists
- the `A` record points to the purchased IPv4 address
- the DNS zone is active
- the domain uses the correct nameservers

HTTPS Certificate Fails

Check:

- `HETZNER_API_TOKEN` is set in GitHub Actions secrets
- the token has permission to update the DNS zone
- DNS records point to the Hetzner server
- `/opt/lounastutka/letsencrypt/acme.json` exists
- `acme.json` has permission mode `600`

Then check Traefik logs:

```
docker service logs -f lounastutka_traefik
```

Stack Is Running But Site Is Down

Check the Swarm services:

```
docker stack services lounastutka
```

Check Traefik:

```
docker service ps lounastutka_traefik
docker service logs -f lounastutka_traefik
```

Check the routed application services:

```
docker service ps lounastutka_frontend
docker service ps lounastutka_app
docker service ps lounastutka_microservice
```

Docker

Lounastutka is designed to run as containers in both development and deployment environments. Docker is used for three main workflows:

- local development with `compose.dev.yaml`
- local production-style testing with `compose.local.swarm.yaml`
- production deployment with Docker Swarm using `compose.yaml`

The Docker setup keeps the application services, database, reverse proxy, and observability tools reproducible across machines.

Docker Files

File	Purpose
`compose.dev.yaml`	Local development environment with live reload.
`compose.local.swarm.yaml`	Local single-node Swarm environment using locally built images.
`compose.yaml`	Production Swarm stack deployed by GitHub Actions.
`Backend/Dockerfile`	Builds the Bun backend into a compiled runtime binary.
`Frontend/Dockerfile`	Builds the frontend and serves it with Nginx.
`microservice/Dockerfile`	Runs the Python scraper microservice with Uvicorn.

Services

The production stack contains these services:

Service	Description
`traefik`	Edge reverse proxy, HTTPS entrypoint, routing, TLS certificates, and dashboard.
`frontend`	Static frontend served through Nginx.
`app`	Backend API service listening internally on port `3001`.
`microservice`	Python scraping service listening internally on port `8100`.

Service	Description
`db`	PostgreSQL database.
`prometheus`	Metrics collection.
`loki`	Log storage.
`promtail`	Docker log collector for Loki.
`grafana`	Observability dashboard.
`portainer`	Docker Swarm management UI.
`whoami`	Small Traefik test service.

The local development stack is smaller and starts only Traefik, frontend, backend, database, and `whoami`.

Local Development

Use the development compose file when actively writing code. It mounts the local source folders into containers and runs the frontend and backend in watch mode.

Start the development environment from the repository root:

```
docker compose -f compose.dev.yaml up
```

Development endpoints:

Service	URL
Frontend	`http://localhost:5173`
Backend API	`http://localhost:3001`
PostgreSQL	`localhost:5432`
Traefik dashboard	`https://dashboard.docker.localhost`
Whoami test service	`https://whoami.docker.localhost`

The backend and frontend containers use the official `oven/bun:1` image in development. Dependencies are installed inside named Docker volumes:

- `backend_node_modules`
- `frontend_node_modules`

This avoids writing container-installed dependencies directly into the host working tree.

Stop the development environment:

```
docker compose -f compose.dev.yaml down
```

Stop and remove local development volumes:

```
docker compose -f compose.dev.yaml down -v
```

Removing volumes resets the local database and dependency volumes.

Local Swarm

Use the local Swarm compose file when you want to test the production deployment shape without pushing images to GitHub Container Registry.

Build the local images:

```
docker build -t lounastutka-backend:local ./Backend
docker build -t lounastutka-frontend:local ./Frontend
docker build -t lounastutka-microservice:local ./microservice
```

Initialize Swarm once:

```
docker swarm init
```

Deploy the local stack:

```
docker stack deploy -c compose.local.swarm.yaml lounastutka
```

Local Swarm endpoints:

Service	URL
Frontend	`http://localhost`
API	`http://localhost/api`
Scraper microservice	`http://localhost/scrape`
Traefik dashboard	`http://dashboard.localhost`

Service	URL
Grafana	`http://grafana.localhost`
Prometheus	`http://prometheus.localhost`
Portainer	`http://portainer.localhost`
Whoami test service	`http://whoami.localhost`

Inspect the running stack:

```
docker stack services lounastutka
docker service ps lounastutka_app
docker service ps lounastutka_frontend
docker service ps lounastutka_microservice
```

Remove the local Swarm stack:

```
docker stack rm lounastutka
```

Production Swarm

Production uses `compose.yaml` and is deployed as a Docker Swarm stack named `lounastutka`.

Images are built by GitHub Actions and pushed to GitHub Container Registry:

- `ghcr.io/<owner>/<repo>/backend:<tag>`
- `ghcr.io/<owner>/<repo>/frontend:<tag>`
- `ghcr.io/<owner>/<repo>/microservice:<tag>`

Each image is tagged with:

- `latest`
- the Git commit SHA

The deployment workflow copies the stack configuration to `/opt/lounastutka` on the server and runs:

```
docker stack deploy -c compose.yaml lounastutka --with-registry-auth
```

The following environment variables are passed during deployment:

Variable	Purpose
`IMAGE_TAG`	Image tag to deploy, normally the Git commit SHA.
`GITHUB_REPOSITORY`	Repository path used to resolve GHCR image names.
`POSTGRES_PASSWORD`	PostgreSQL password.
`POSTGRES_URL`	Backend database connection string.
`JWT_SECRET`	Backend JWT signing secret.
`JWT_EXPIRES_IN`	JWT expiry duration, defaults to `1d`.
`WEBAUTHN_RP_ID`	WebAuthn relying party ID.
`WEBAUTHN_RP_NAME`	WebAuthn relying party display name.
`WEBAUTHN_ORIGIN`	Allowed WebAuthn origin.
`HETZNER_API_TOKEN`	Hetzner DNS token used by Traefik for ACME DNS challenge.
`GF_ADMIN_PASSWORD`	Grafana admin password.

Production routes are handled by Traefik:

Route	Service
`https://lounastutka.fi`	Frontend
`https://lounastutka.fi/api`	Backend API
`https://lounastutka.fi/scrape`	Scraper microservice
`https://dashboard.swarm.lounastutka.fi`	Traefik dashboard
`https://grafana.lounastutka.fi`	Grafana
`https://prometheus.swarm.lounastutka.fi`	Prometheus
`https://portainer.lounastutka.fi`	Portainer
`https://whoami.swarm.lounastutka.fi`	Traefik test service

Images

Backend

`Backend/Dockerfile` uses a multi-stage build:

1. installs dependencies with Bun

2. compiles `index.ts` into a standalone binary
3. copies the binary into a minimal Debian runtime image

The final container exposes port `3001`.

Frontend

`Frontend/Dockerfile` uses a multi-stage build:

1. installs frontend dependencies with npm
2. builds the application
3. copies the built client files into an Nginx image

Nginx uses `Frontend/docker/nginx/default.conf` and serves the frontend on port `80`.

Microservice

`microservice/Dockerfile` uses Python `3.11-slim`, installs dependencies with `uv`, and starts Uvicorn:

```
uvicorn scraper:app --host 0.0.0.0 --port 8100
```

The final container exposes the application internally on port `8100`.

Networks

The Swarm stack uses overlay networks:

Network	Purpose
`traefik_proxy`	Public application routing through Traefik.
`backend`	Internal application and database communication.
`monitoring`	Prometheus, Loki, Promtail, Grafana, and Traefik metrics.
`edge`	Portainer and Portainer agent communication.

The development compose file uses a local Docker network named `proxy`.

Volumes

Important production volumes:

Volume	Purpose
`pg_data`	PostgreSQL data.
`prom_data`	Prometheus data.
`grafana_data`	Grafana data.
`portainer_data`	Portainer data.
`./letsencrypt`	Traefik ACME certificate storage.
`./Database/init`	SQL initialization scripts mounted into PostgreSQL.
`./docker/grafana/provisioning`	Grafana datasource and dashboard provisioning.
`./docker/prometheus/prometheus.yml`	Prometheus configuration.
`./docker/promtail/config.yaml`	Promtail configuration.

PostgreSQL, Grafana, Prometheus, and Portainer store persistent data in Docker volumes. Removing these volumes deletes their stored state.

Useful Commands

Show local compose containers:

```
docker compose -f compose.dev.yaml ps
```

Follow local development logs:

```
docker compose -f compose.dev.yaml logs -f
```

Show Swarm services:

```
docker stack services lounastutka
```

Show tasks for a service:

```
docker service ps lounastutka_app
```

Follow Swarm service logs:

```
docker service logs -f lounastutka_app
docker service logs -f lounastutka_frontend
docker service logs -f lounastutka_microservice
docker service logs -f lounastutka_traefik
```

Remove the Swarm stack:

```
docker stack rm lounastutka
```

Leave Swarm mode:

```
docker swarm leave --force
```

Troubleshooting

Port Already in Use

Local development binds ports `80`, `443`, `3001`, `5173`, and `5432`. Stop the conflicting local process or edit the port mapping in `compose.dev.yaml`.

Database Does Not Initialize Again

PostgreSQL only runs scripts from `Database/init` when the data directory is empty. To recreate the local development database:

```
docker compose -f compose.dev.yaml down -v
docker compose -f compose.dev.yaml up
```

Swarm Service Does Not Update

Check which image tag is deployed:

```
docker service inspect lounastutka_app --format '{{.Spec.TaskTemplate.ContainerSpec.Image}}'
```

Then inspect service tasks and logs:

```
docker service ps lounastutka_app
docker service logs -f lounastutka_app
```

Traefik Cannot Route to a Service

Check that the service has:

- `traefik.enable=true`
- a router rule
- the correct Traefik entrypoint
- the correct internal service port
- membership in the `traefik_proxy` network

In production Swarm, Traefik is configured to use the `lounastutka_traefik_proxy` network.

TLS Certificate Issues

Production certificates are managed by Traefik using the Hetzner DNS challenge. Verify that:

- `HETZNER_API_TOKEN` is set
- `/opt/lounastutka/letsencrypt/acme.json` exists
- `acme.json` has permission mode `600`
- DNS records point to the Swarm host

The GitHub Actions deploy step creates `letsencrypt/acme.json` automatically before deploying the stack.

GitHub Actions

Lounastutka uses GitHub Actions to build Docker images and deploy the production Docker Swarm stack. The workflow is defined in:

```
.github/workflows/deploy.yml
```

The workflow is named `Build & Deploy`.

Purpose

The workflow handles two responsibilities:

- building the backend, frontend, and microservice Docker images
- deploying `compose.yaml` to the production server with Docker Swarm

Images are stored in GitHub Container Registry, using the `ghcr.io` registry.

Triggers

The workflow runs on:

Trigger	Behavior
Pull request opened against `main`	Builds images for validation, but does not push them.
Pull request synchronized against `main`	Rebuilds images after new commits, but does not push th
Pull request reopened against `main`	Builds images for validation, but does not push them.
Pull request closed and merged into `main`	Builds, pushes, and deploys.
Manual `workflow_dispatch`	Builds, pushes, and deploys on demand.

The workflow does not deploy every pull request. Deployment only happens when a pull request is merged or when the workflow is started manually.

Jobs

The workflow has two jobs:

Job	Purpose
`build-and-push`	Builds Docker images and pushes them when deployment is allowed.
`deploy`	Copies configuration to the server and runs `docker stack deploy`.

Build And Push Job

The `build-and-push` job runs on `ubuntu-latest`.

It has these permissions:

Permission	Reason
`contents: read`	Allows the workflow to check out the repository.
`packages: write`	Allows pushing images to GitHub Container Registry.

Repository Name Normalization

The workflow converts `GITHUB_REPOSITORY` to lowercase:

```
echo "repo=${GITHUB_REPOSITORY,,}" >> "$GITHUB_OUTPUT"
```

This is needed because container registry image names must be lowercase. The normalized repository name is exposed as the `image_repo` job output and reused during deployment.

Registry Login

The job logs in to GitHub Container Registry:

```
ghcr.io
```

Authentication uses:

Value	Source
Username	`${{ github.actor }}`
Password	`${{ secrets.GITHUB_TOKEN }}`

`GITHUB_TOKEN` is provided automatically by GitHub Actions.

The job also logs in to Docker Hub to avoid anonymous pull rate limits when BuildKit pulls base images.

Buildx

The workflow enables Docker Buildx with:

```
docker/setup-buildx-action@v4
```

Buildx is used by the `docker/build-push-action` steps.

Built Images

The workflow builds three images:

Image	Build context	Platform
`backend`	`./Backend`	`linux/amd64`
`frontend`	`./Frontend`	`linux/amd64`
`microservice`	`./microservice`	`linux/amd64`

Each image is tagged with:

Tag	Purpose
`latest`	Convenient reference to the most recent deployed image.
`${{ github.sha }}`	Immutable image tag for the exact commit.

The final image names follow this pattern:

```
ghcr.io/<owner>/<repo>/backend:<tag>
ghcr.io/<owner>/<repo>/frontend:<tag>
ghcr.io/<owner>/<repo>/microservice:<tag>
```

For example:

```
ghcr.io/example/lounastutka/backend:latest
ghcr.io/example/lounastutka/backend:<commit-sha>
```

Push Rules

Images are pushed only when:

- the workflow was started manually with `workflow_dispatch`
- a pull request was merged

For normal pull request updates, the images are built but not pushed.

Build Cache

Each image uses GitHub Actions cache with a separate scope:

Image	Cache scope
Backend	`backend`
Frontend	`frontend`
Microservice	`microservice`

The workflow reads from and writes to the cache:

```
cache-from: type=gha,scope=<service>
cache-to: type=gha,mode=max,scope=<service>
```

This speeds up repeated builds.

Deploy Job

The `deploy` job depends on `build-and-push`.

It runs only when:

- the workflow was started manually
- a pull request was merged

The deploy job performs two steps:

1. Copy configuration files to the production server.
2. SSH into the server and deploy the Docker Swarm stack.

Files Copied To The Server

The workflow uses `appleboy/scp-action` to copy:

```
compose.yaml
docker/
Database/
```

The files are copied to:

```
/opt/lounastutka
```

These files are required because:

Path	Purpose
`compose.yaml`	Production Docker Swarm stack definition.
`docker/`	Prometheus, Promtail, and Grafana configuration.
`Database/`	Database initialization SQL files.

Stack Deployment

The workflow uses `appleboy/ssh-action` to run deployment commands on the server.

Before deploying, it prepares the Traefik ACME certificate file:

```
mkdir -p /opt/lounastutka/letsencrypt
touch /opt/lounastutka/letsencrypt/acme.json
chmod 600 /opt/lounastutka/letsencrypt/acme.json
```

Then it logs in to GitHub Container Registry from the server:

```
echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.actor }} --password-
```

Finally, it deploys the stack:

```
docker stack deploy -c compose.yaml lounastutka --with-registry-auth
```

The `--with-registry-auth` flag forwards registry credentials to the Swarm so worker nodes can pull private images.

Deployment Environment Variables

The deploy step passes environment variables directly into the `docker stack deploy` command.

Variable	Purpose
`IMAGE_TAG`	The commit SHA image tag to deploy.
`GITHUB_REPOSITORY`	Lowercase repository path used in GHCR image names.
`POSTGRES_PASSWORD`	PostgreSQL password.
`POSTGRES_URL`	Backend database connection string.
`JWT_SECRET`	Backend JWT signing secret.
`WEBAUTHN_RP_ID`	WebAuthn relying party ID.
`WEBAUTHN_RP_NAME`	WebAuthn relying party name.
`WEBAUTHN_ORIGIN`	Allowed WebAuthn origin.
`HETZNER_API_TOKEN`	Hetzner DNS API token used by Traefik ACME DNS challenge.
`GF_ADMIN_PASSWORD`	Grafana admin password.

These values are consumed by `compose.yaml`.

Required Secrets

The workflow depends on these GitHub Actions secrets:

Secret	Used by	Purpose
`HETZNER_HOST`	Deploy job	Production server hostname or IP address.

Secret	Used by	Purpose
`HETZNER_USER`	Deploy job	SSH user used for deployment.
`HETZNER_SSH_KEY`	Deploy job	Private SSH key for the deployment user.
`POSTGRES_PASSWORD`	Deploy job	Production PostgreSQL password.
`POSTGRES_URL`	Deploy job	Backend database connection string.
`JWT_SECRET`	Deploy job	JWT signing secret.
`WEBAUTHN_RP_ID`	Deploy job	WebAuthn relying party ID.
`WEBAUTHN_RP_NAME`	Deploy job	WebAuthn relying party display name.
`WEBAUTHN_ORIGIN`	Deploy job	WebAuthn origin.
`HETZNER_API_TOKEN`	Deploy job	DNS token for Traefik certificate automation.
`GF_ADMIN_PASSWORD`	Deploy job	Grafana admin password.
`DOCKERHUB_USERNAME`	Build job	Docker Hub username for authenticated pulls.
`DOCKERHUB_TOKEN`	Build job	Docker Hub token for authenticated pulls.

`GITHUB_TOKEN` is provided automatically by GitHub Actions and should not be added manually.

Deployment Flow

The normal production deployment flow is:

1. A pull request is merged into `main`.
2. GitHub Actions checks out the repository.
3. The workflow logs in to GHCR and Docker Hub.
4. Buildx builds the backend, frontend, and microservice images.
5. Images are pushed to GHCR with `latest` and commit SHA tags.
6. The deploy job copies `compose.yaml`, `docker/`, and `Database/` to `/opt/lounastutka`.
7. The server logs in to GHCR.
8. Docker Swarm deploys the `lounastutka` stack using the new image tag.

Manual Deployment

The workflow can also be started manually from the GitHub Actions UI using `workflow_dispatch`.

Manual deployment is useful when:

- redeploying the current commit
- recovering from a failed server-side deploy
- verifying deployment changes without merging a new pull request

Manual runs build, push, and deploy just like a merged pull request.

Production Server Requirements

The production server must have:

- Docker Engine installed
- Docker Swarm initialized
- SSH access for the deployment user
- access to `/opt/lounastutka`
- permission to run Docker commands
- DNS records pointing to the server

The deployment user should be able to run:

```
docker stack deploy
docker service ls
docker service ps
docker service logs
```

Verifying A Deployment

After a deployment, check the stack:

```
docker stack services lounastutka
```

Check backend tasks:

```
docker service ps lounastutka_app
```

Check frontend tasks:

```
docker service ps lounastutka_frontend
```

Check microservice tasks:

```
docker service ps lounastutka_microservice
```

Follow logs:

```
docker service logs -f lounastutka_app
docker service logs -f lounastutka_frontend
docker service logs -f lounastutka_microservice
docker service logs -f lounastutka_traefik
```

Troubleshooting

Build Runs But Images Are Not Pushed

This is expected for normal pull request builds. Images are pushed only for merged pull requests and manual workflow runs.

Deploy Job Is Skipped

The deploy job runs only when:

- the event is `workflow_dispatch`
- the pull request was closed as merged

If a pull request is closed without merging, deployment is skipped.

GHCR Push Fails

Check that the workflow has:

```yaml
permissions:
  contents: read
  packages: write
```

Also check repository package permissions if the image package already exists in GitHub Container Registry.

Docker Hub Login Fails

Check these secrets:

- `DOCKERHUB_USERNAME`

- `DOCKERHUB_TOKEN`

This login is used to avoid Docker Hub pull rate limits while building images. It is separate from GHCR login.

SSH Or SCP Fails

Check these secrets:

- `HETZNER_HOST`
- `HETZNER_USER`
- `HETZNER_SSH_KEY`

Also verify that the public key matching `HETZNER_SSH_KEY` exists in the deployment user's `authorized_keys` file on the server.

Stack Deploy Fails To Pull Images

Check that:

- the images were pushed successfully to GHCR
- the server can log in to `ghcr.io`
- `docker stack deploy` includes `--with-registry-auth`
- `GITHUB_REPOSITORY` is lowercase in the deployed environment
- `IMAGE_TAG` matches an image tag that exists in GHCR

Traefik Certificate Creation Fails

Check that:

- `HETZNER_API_TOKEN` is configured
- `/opt/lounastutka/letsencrypt/acme.json` exists
- `acme.json` has permission mode `600`
- the domain DNS records point to the server

The workflow creates the ACME file before each deployment, but DNS and token permissions must still be valid.

Traefik

Traefik is the reverse proxy for Lounastutka. It receives HTTP and HTTPS traffic, discovers Docker services, and routes requests to the correct container based on hostnames and paths.

The project uses Traefik in three environments:

- local development with `compose.dev.yaml`
- local Swarm testing with `compose.local.swarm.yaml`
- production Swarm deployment with `compose.yaml`

Role In The System

Traefik is responsible for:

- exposing the public entrypoints on ports `80` and `443`
- redirecting HTTP traffic to HTTPS in environments that use TLS
- terminating TLS certificates
- routing frontend requests to the frontend service
- routing API requests to the backend service
- routing scraper requests to the microservice
- exposing dashboards for Traefik and monitoring tools
- exporting access logs and Prometheus metrics

Services are not exposed automatically. They must opt in with Traefik labels.

Configuration Style

Traefik configuration is split into two parts:

Type	Location	Purpose
Static configuration	`command:` arguments on the `traefik` service	Entrypoints, providers, logging, metrics, cert
Dynamic configuration	Docker or Swarm labels	Routers, services, middlewares, TLS flags, hc

Local development also mounts:

```
dynamic/tls.yaml
certs/
```

This provides a local certificate for HTTPS development.

Entrypoints

Production

Production Traefik exposes:

Entrypoint	Port	Purpose
`web`	`80`	HTTP entrypoint. Redirects to HTTPS.
`websecure`	`443`	HTTPS entrypoint for application traffic.
`metrics`	`8082`	Internal Prometheus metrics entrypoint.

The production stack publishes ports `80` and `443` in host mode:

```yaml
ports:
  - target: 80
    published: 80
    mode: host
  - target: 443
    published: 443
    mode: host
```

HTTP is redirected permanently to HTTPS:

```
--entrypoints.web.http.redirections.entrypoint.to=websecure
--entrypoints.web.http.redirections.entrypoint.scheme=https
--entrypoints.web.http.redirections.entrypoint.permanent=true
```

Local Swarm

Local Swarm exposes:

Entrypoint	Port	Purpose
`web`	`80`	Local HTTP routes.

Entrypoint	Port	Purpose
`traefik`	`8080`	Dashboard and metrics entrypoint.

Local Swarm does not configure TLS. It is intended for testing the Swarm routing shape locally.

Local Development

Local development exposes:

Entrypoint	Port	Purpose
`web`	`80`	HTTP entrypoint. Redirects to HTTPS.
`websecure`	`443`	HTTPS entrypoint using local certificates.

The development setup uses `dynamic/tls.yaml`:

```yaml
tls:
  certificates:
    - certFile: /certs/local.crt
      keyFile:  /certs/local.key
```

The certificate and key are mounted from:

```
certs/local.crt
certs/local.key
```

Providers

Production And Local Swarm

Production and local Swarm use the Docker Swarm provider:

```
--providers.swarm.endpoint=unix:///var/run/docker.sock
--providers.swarm.watch=true
--providers.swarm.exposedbydefault=false
--providers.swarm.network=lounastutka_traefik_proxy
```

The Docker socket is mounted read-only:

```yaml
volumes:
  - /var/run/docker.sock:/var/run/docker.sock:ro
```

Traefik must run on a manager node because it reads Swarm service metadata through the Docker socket.

Local Development

Local development uses the Docker provider:

```
--providers.docker=true
--providers.docker.exposedbydefault=false
--providers.docker.network=proxy
```

The development network is named:

```
proxy
```

Networks

In Swarm, public services that Traefik routes to must be connected to:

```
traefik_proxy
```

The fully qualified Swarm network name is:

```
lounastutka_traefik_proxy
```

Traefik also connects to the `monitoring` network so Prometheus can reach metrics.

Services that are internal only, such as PostgreSQL and Loki, do not need Traefik labels.

Production Routes

Production routing is defined by labels in `compose.yaml`.

Route	Router	Target service	Internal port
`https://lounastutka.fi`	`frontend`	`frontend`	`80`

Route	Router	Target service	Internal port
`https://lounastutka.fi/api`	`api`	`app`	`3001`
`https://lounastutka.fi/scrape`	`microservice`	`microservice`	`8100`
`https://dashboard.swarm.lounastutka.fi`	`dashboard`	`api@internal`	internal
`https://whoami.swarm.lounastutka.fi`	`whoami`	`whoami`	`80`
`https://prometheus.swarm.lounastutka.fi`	`prometheus`	`prometheus`	`9090`
`https://grafana.lounastutka.fi`	`grafana`	`grafana`	`3000`
`https://portainer.lounastutka.fi`	`portainer`	`portainer`	`9000`

The API router uses a path prefix:

```
Host(`lounastutka.fi`) && PathPrefix(`/api`)
```

The scraper microservice router also uses a path prefix:

```
Host(`lounastutka.fi`) && PathPrefix(`/scrape`)
```

The frontend router matches the root host:

```
Host(`lounastutka.fi`)
```

Local Swarm Routes

Local Swarm routes are defined in `compose.local.swarm.yaml`.

Route	Target service	Internal port
`http://localhost`	`frontend`	`80`
`http://localhost/api`	`app`	`3001`
`http://localhost/scrape`	`microservice`	`8100`
`http://dashboard.localhost`	Traefik dashboard	`8080`
`http://whoami.localhost`	`whoami`	`80`
`http://prometheus.localhost`	`prometheus`	`9090`
`http://grafana.localhost`	`grafana`	`3000`

Route	Target service	Internal port
`http://portainer.localhost`	`portainer`	`9000`

Local Development Routes

Local development routes are defined in `compose.dev.yaml`.

Route	Target service
`https://localhost`	`frontend`
`https://localhost/api`	`backend`
`https://dashboard.docker.localhost`	Traefik dashboard
`https://whoami.docker.localhost`	`whoami`

The frontend and backend also publish direct development ports:

Service	Direct URL
Frontend Vite dev server	`http://localhost:5173`
Backend API	`http://localhost:3001`

These direct ports are useful during development, while the Traefik routes are useful for testing proxy behavior.

Labels

A routed service needs these labels:

```
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.<router-name>.rule=Host(`<host>`)"
  - "traefik.http.routers.<router-name>.entrypoints=<entrypoint>"
  - "traefik.http.services.<service-name>.loadbalancer.server.port=<internal-port>"
```

Production HTTPS routes also include:

```
  - "traefik.http.routers.<router-name>.tls=true"
  - "traefik.http.routers.<router-name>.tls.certresolver=le"
```

In Swarm mode, labels belong under `deploy.labels`, not top-level service `labels`.

TLS Certificates

Production Certificates

Production uses Let's Encrypt certificates through Traefik's ACME DNS challenge.

The certificate resolver is named:

```
le
```

The production resolver uses Hetzner DNS:

```
--certificatesresolvers.le.acme.dnschallenge=true
--certificatesresolvers.le.acme.dnschallenge.provider=hetzner
```

The Hetzner API token is passed to the Traefik container as:

```
HETZNER_API_TOKEN
```

Certificate state is stored in:

```
/letsencrypt/acme.json
```

On the host this is mounted from:

```
/opt/lounastutka/letsencrypt/acme.json
```

The GitHub Actions deployment creates this file before deploying:

```
mkdir -p /opt/lounastutka/letsencrypt
touch /opt/lounastutka/letsencrypt/acme.json
chmod 600 /opt/lounastutka/letsencrypt/acme.json
```

Local Development Certificates

Local development uses the file provider and local certificate files:

```
--providers.file.filename=/dynamic/tls.yaml
```

The local certificate files are:

```
certs/local.crt
certs/local.key
```

Browsers may show a warning unless the local certificate is trusted by the operating system.

Dashboard

The Traefik dashboard is enabled in all environments:

```
--api.dashboard=true
--api.insecure=false
```

Because insecure mode is disabled, the dashboard is exposed through a normal Traefik router using:

```
api@internal
```

Dashboard routes:

Environment	URL
Production	`https://dashboard.swarm.lounastutka.fi`
Local Swarm	`http://dashboard.localhost`
Local development	`https://dashboard.docker.localhost`

Production and local development protect the dashboard with basic authentication.

Middlewares

The current configuration defines basic-auth middlewares for protected dashboards.

Middleware	Environment	Used by
`dashboard-auth@swarm`	Production Swarm	Traefik dashboard
`prometheus-auth@swarm`	Production Swarm	Prometheus

Middleware	Environment	Used by
`dashboard-auth@docker`	Local development	Traefik dashboard

The password hashes are stored in compose labels. In YAML, dollar signs are escaped as `$$`.

Logging And Metrics

Traefik access logs are enabled:

```
--accesslog=true
```

Traefik Prometheus metrics are enabled:

```
--metrics.prometheus=true
```

In production, metrics are served on the internal `metrics` entrypoint:

```
--metrics.prometheus.entrypoint=metrics
--entrypoints.metrics.address=:8082
```

Local Swarm attaches metrics to the `traefik` entrypoint on port `8080`.

Useful Commands

Show the Traefik service:

```
docker service ps lounastutka_traefik
```

Follow Traefik logs:

```
docker service logs -f lounastutka_traefik
```

Inspect the production Traefik service:

```
docker service inspect lounastutka_traefik
```

List stack services:

```
docker stack services lounastutka
```

For local development logs:

```
docker compose -f compose.dev.yaml logs -f traefik
```

Adding A New Routed Service

To expose a new service through Traefik:

1. Connect the service to the Traefik network.
2. Add `traefik.enable=true`.
3. Add a router rule with `Host(...)` and, if needed, `PathPrefix(...)`.
4. Set the router entrypoint.
5. Set TLS labels for production HTTPS routes.
6. Set the load balancer port to the container's internal port.

Example Swarm labels:

```yaml
deploy:
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.example.rule=Host(`lounastutka.fi`) && PathPrefix(`/example`)"
    - "traefik.http.routers.example.entrypoints=websecure"
    - "traefik.http.routers.example.tls=true"
    - "traefik.http.routers.example.tls.certresolver=le"
    - "traefik.http.services.example.loadbalancer.server.port=8080"
```

Troubleshooting

Service Is Not Visible In Traefik

Check that:

- the service has `traefik.enable=true`
- Swarm labels are under `deploy.labels`
- the service is connected to `traefik_proxy`
- Traefik is using `lounastutka_traefik_proxy`
- the service has at least one running task

Route Returns 404

A Traefik 404 usually means no router matched the request.

Check:

- the hostname in the router rule
- the path prefix in the router rule
- whether the request is using HTTP or HTTPS
- whether the router is attached to the correct entrypoint

Route Returns Bad Gateway

A bad gateway usually means Traefik matched the router but could not reach the target service.

Check:

- the target service is running
- the internal load balancer port is correct
- the service and Traefik share a network
- the application is listening on `0.0.0.0`, not only `localhost`

TLS Certificate Is Not Created

Check that:

- `HETZNER_API_TOKEN` is set
- DNS records point to the production server
- `/opt/lounastutka/letsencrypt/acme.json` exists
- `acme.json` has permission mode `600`
- Traefik can write to the mounted `letsencrypt` directory

Then inspect Traefik logs:

```
docker service logs -f lounastutka_traefik
```

Dashboard Requires Login

The production and development dashboards use basic authentication. The credentials are configured in Traefik middleware labels in the compose files.

If login fails, update the basic-auth hash in the relevant compose file and redeploy the stack.

Nginx

Nginx is used as the production web server for the Lounastutka frontend. It serves the static frontend files that are produced by the Vite build and runs inside the frontend Docker image.

Nginx is not used for active local frontend development. In local development, the frontend runs with the Vite dev server inside the `oven/bun:1` container.

Role In The System

Nginx is responsible for:

- serving the built frontend application
- returning `index.html` for client-side routes
- caching hashed static assets
- forwarding selected frontend-origin API paths to the backend service

Traefik is still the public edge reverse proxy. In production, requests flow like this:

```
User -> Traefik -> frontend service -> Nginx -> static frontend files
```

For backend traffic, Traefik normally routes `/api` directly to the backend service:

```
User -> Traefik -> backend service
```

The Nginx config also contains API proxy rules for `/api` and `/auth`. These are useful if a request reaches the frontend container first and then needs to be forwarded internally to the backend.

Where Nginx Is Configured

The Nginx configuration file is:

```
Frontend/docker/nginx/default.conf
```

The frontend Dockerfile copies it into the Nginx image:

```
COPY docker/nginx/default.conf /etc/nginx/conf.d/default.conf
```

The built frontend files are copied to:

```
/usr/share/nginx/html
```

Frontend Docker Image

The frontend image is built with `Frontend/Dockerfile`.

It has three stages:

Stage	Base image	Purpose
`deps`	`node:22-alpine`	Installs frontend dependencies.
`build`	`node:22-alpine`	Builds the frontend with Vite.
final	`nginx:1.27-alpine`	Serves the built frontend files.

The build command is:

```
npm run build
```

The `build` script runs:

```
vite build
```

Vite writes the production client bundle to:

```
build/client
```

The final Docker image copies that output into Nginx:

```
COPY --from=build /app/build/client /usr/share/nginx/html
```

Server Block

The Nginx server listens on port `80`:

```
server {
    listen 80;
    server_name _;
}
```

The wildcard `server_name _;` is used because Traefik handles the public hostname routing before traffic reaches the frontend container.

In Docker Swarm, Traefik routes frontend traffic to this internal port:

```
traefik.http.services.frontend.loadbalancer.server.port=80
```

Static File Root

Nginx serves files from:

```
root /usr/share/nginx/html;
index index.html;
```

This directory contains the built Vite application.

SPA Routing

The frontend is a single-page application. Browser routes such as `/settings` or `/components` are handled by the frontend router, not by separate files on disk.

Nginx supports this with:

```
location / {
    try_files $uri $uri/ /index.html;
}
```

This means:

1. Try to serve the requested file.
2. Try to serve the requested directory.
3. Fall back to `index.html`.

The fallback allows direct page refreshes and deep links to work.

Static Asset Caching

Vite outputs static assets with content hashes in their filenames. Because the filename changes when the content changes, these files can be cached aggressively.

Nginx configures long-lived caching for:

```
location /assets/ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
```

This improves frontend load times while still allowing safe cache busting when a new frontend build is deployed.

API Proxy Rules

The Nginx config contains proxy rules for:

```
/auth
/api
```

Both are forwarded to the backend service:

```
proxy_pass http://app:3001;
```

In the production Swarm stack, the backend service is named:

```
app
```

It listens internally on port:

```
3001
```

The proxy configuration also forwards upgrade headers:

```
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection 'upgrade';
proxy_set_header Host $host;
proxy_cache_bypass $http_upgrade;
```

This preserves the original host and supports upgraded HTTP connections if the backend needs them.

Production Routing

In production, Traefik exposes the frontend service at:

```
https://lounastutka.fi
```

The frontend service uses the image:

```
ghcr.io/${GITHUB_REPOSITORY}/frontend:${IMAGE_TAG:-latest}
```

The service is connected to the `traefik_proxy` network and Traefik routes to Nginx on port `80`.

Production frontend labels:

```
- traefik.enable=true
- traefik.http.routers.frontend.rule=Host(`lounastutka.fi`)
- traefik.http.routers.frontend.entrypoints=websecure
- traefik.http.routers.frontend.tls=true
- traefik.http.routers.frontend.tls.certresolver=le
- traefik.http.services.frontend.loadbalancer.server.port=80
```

Local Swarm Routing

In local Swarm, the frontend image is:

```
lounastutka-frontend:local
```

Build it from the repository root:

```
docker build -t lounastutka-frontend:local ./Frontend
```

The frontend is available through Traefik at:

```
http://localhost
```

Local Swarm frontend labels:

```
- "traefik.enable=true"
- "traefik.http.routers.frontend.rule=Host(`localhost`)"
- "traefik.http.routers.frontend.entrypoints=web"
- "traefik.http.services.frontend.loadbalancer.server.port=80"
```

Local Development

Local development does not use the Nginx frontend image.

Instead, `compose.dev.yaml` runs the frontend with:

```
bun install && bun run dev -- --host 0.0.0.0
```

The development frontend is available directly at:

```
http://localhost:5173
```

It is also routed through Traefik at:

```
https://localhost
```

Use the Nginx image when testing production-like behavior, especially SPA fallback and static asset caching.

Useful Commands

Build the production frontend image locally:

```
docker build -t lounastutka-frontend:local ./Frontend
```

Run the local Swarm stack:

```
docker stack deploy -c compose.local.swarm.yaml lounastutka
```

Check the frontend service:

```
docker service ps lounastutka_frontend
```

Follow frontend service logs:

```
docker service logs -f lounastutka_frontend
```

Inspect the frontend image:

```
docker image inspect lounastutka-frontend:local
```

Troubleshooting

Page Refresh Returns 404

Check the SPA fallback:

```
location / {
    try_files $uri $uri/ /index.html;
}
```

If this fallback is missing, direct navigation to frontend routes may fail.

Static Assets Do Not Load

Check that the Vite build output exists in the image:

```
/usr/share/nginx/html
```

Also check that `Frontend/vite.config.ts` still builds to:

```
build/client
```

The Dockerfile expects that path.

Frontend Service Returns Bad Gateway

If Traefik returns a bad gateway for the frontend route, check:

- the frontend service is running

- the frontend service is attached to `traefik_proxy`
- Traefik routes to port `80`
- Nginx is listening on port `80`

Useful commands:

```
docker service ps lounastutka_frontend
docker service logs -f lounastutka_frontend
docker service logs -f lounastutka_traefik
```

API Calls Fail From The Frontend

Check whether the request is intended to go through Traefik or Nginx.

In production, `/api` is normally routed by Traefik directly to the backend service. If a request reaches the frontend container first, Nginx proxies `/api` and `/auth` to:

```
http://app:3001
```

If that proxy path fails, check that:

- the backend service is named `app`
- the backend listens on port `3001`
- the frontend and backend share a Docker network where `app` resolves

Old Frontend Assets Are Still Loaded

The `/assets/` directory is cached for one year because Vite assets are content-hashed.

If stale assets appear:

- verify the new build produced new hashed filenames
- verify the deployed image tag changed
- hard-refresh the browser
- check whether a CDN or browser cache is serving old HTML

The HTML document itself should not be cached as aggressively as hashed assets.

PostgreSQL

PostgreSQL is the relational database for Lounastutka. It stores user, authentication, restaurant, menu, and menu item data.

The project runs PostgreSQL in all Docker environments:

- local development with `compose.dev.yaml`
- local Swarm testing with `compose.local.swarm.yaml`
- production Swarm deployment with `compose.yaml`

The Docker service is named `db`.

Role In The System

PostgreSQL is responsible for:

- storing application user accounts
- storing passkey authentication data
- storing temporary authentication challenges
- storing restaurants
- storing menus and menu items
- providing the backend API with persistent relational data

The backend connects to PostgreSQL through the `POSTGRES_URL` connection string.

Service Configuration

Production uses the `postgres:16-alpine` image:

```yaml
db:
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: ${POSTGRES_USER:-lounastutka}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    POSTGRES_DB: ${POSTGRES_DB:-lounastutka}
  volumes:
    - pg_data:/var/lib/postgresql/data
    - ./Database/init:/docker-entrypoint-initdb.d:ro
  networks:
    - backend
```

The database listens internally on port `5432`.

Environment Variables

The PostgreSQL service uses:

Variable	Purpose	Default
`POSTGRES_USER`	Database user created by the image.	`lounastutka`
`POSTGRES_PASSWORD`	Password for the database user.	No production default
`POSTGRES_DB`	Database name created by the image.	`lounastutka`

The backend uses:

Variable	Purpose
`POSTGRES_URL`	Full PostgreSQL connection string used by the backend API.

Production receives `POSTGRES_PASSWORD` and `POSTGRES_URL` from GitHub Actions secrets during deployment.

Example local connection string:

```
postgres://lounastutka:changeme@db:5432/lounastutka
```

Networks

Production And Local Swarm

In Swarm environments, PostgreSQL is connected only to the internal `backend` network:

```
networks:
  - backend
```

It is not exposed through Traefik and it does not publish port `5432` to the host.

Services that need database access must join the `backend` network. The backend API and scraper microservice are connected to this network.

Local Development

In local development, PostgreSQL is connected to the `proxy` network and publishes port `5432`:

```
ports:
  - "5432:5432"
```

This allows local tools to connect directly to PostgreSQL through:

```
localhost:5432
```

Storage

PostgreSQL stores database files in:

```
/var/lib/postgresql/data
```

Each environment uses a different Docker volume:

Environment	Volume
Production	`pg_data`
Local Swarm	`pg_data_local_swarm`
Local development	`pg_data_dev`

Removing the volume deletes the database data for that environment.

Initialization

The repository stores database initialization SQL in:

```
Database/init
```

The folder is mounted into the PostgreSQL image at:

```
/docker-entrypoint-initdb.d
```

PostgreSQL runs files from this directory only when the database volume is initialized for the first time. If the volume already contains a database, changing `Database/init/schema.sql` does not automatically apply changes.

To apply schema changes to an existing database, use migrations or run SQL manually.

Schema

The current schema file is:

```
Database/init/schema.sql
```

It creates the `app` schema:

```
CREATE SCHEMA IF NOT EXISTS app AUTHORIZATION lounastutka;
```

It creates these tables:

Table	Purpose
`app.users`	User accounts and password hashes.
`app.passkeys`	WebAuthn passkey credentials.
`app.auth_challenges`	Temporary authentication challenges.
`app.restaurants`	Restaurant metadata.
`app.menus`	Menus by restaurant and date.
`app.menu_items`	Individual menu items.

The restaurant, menu, passkey, and challenge tables use foreign keys with `ON DELETE CASCADE` where related rows should be removed with their parent row.

Placement

In Swarm environments, PostgreSQL runs as one replica on a manager node:

```
deploy:
  replicas: 1
  placement:
    constraints:
      - node.role == manager
```

This keeps the database tied to the node that has the expected Docker volume.

Healthcheck

Local development defines a PostgreSQL healthcheck:

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-lounastutka}"]
  interval: 5s
  timeout: 5s
  retries: 5
```

The backend waits for this healthcheck in `compose.dev.yaml`.

The Swarm compose files do not currently define a database healthcheck.

Useful Commands

Show the PostgreSQL service:

```
docker service ps lounastutka_db
```

Follow PostgreSQL logs:

```
docker service logs -f lounastutka_db
```

Inspect the service:

```
docker service inspect lounastutka_db
```

List stack services:

```
docker stack services lounastutka
```

For local development logs:

```
docker compose -f compose.dev.yaml logs -f db
```

Connect from the local development host:

```
psql postgres://lounastutka:changeme@localhost:5432/lounastutka
```

Connect from inside the local development container:

```
docker compose -f compose.dev.yaml exec db psql -U lounastutka -d lounastutka
```

Backups

The production database is stored in the `pg_data` Docker volume on the Hetzner server.

Before destructive changes, create a database dump:

```
docker exec $(docker ps -q -f name=lounastutka_db) pg_dump -U lounastutka lounastutka > lour
```

Also back up the Docker volume or the server state if the deployment depends on local volume persistence.

Troubleshooting

Backend Cannot Connect To Database

Check that:

- the `db` service is running
- the backend and database share the `backend` network in Swarm
- `POSTGRES_URL` points to `db:5432`
- `POSTGRES_PASSWORD` matches the initialized database user password
- the database volume was not initialized with different credentials

Useful commands:

```
docker service logs -f lounastutka_db
docker service logs -f lounastutka_app
```

Password Changes Do Not Work

The official PostgreSQL image uses `POSTGRES_PASSWORD` when initializing a new database directory. If the `pg_data` volume already exists, changing the environment variable does not rewrite the existing user's password.

Update the password inside PostgreSQL or recreate the volume only if losing the data is acceptable.

Schema Changes Do Not Appear

Files in `Database/init` run only during first initialization. If `pg_data` already exists, updates to `schema.sql` will not be applied automatically.

Use migrations or apply SQL manually for existing databases.

Local Port 5432 Is Not Reachable

Port `5432` is published only in `compose.dev.yaml`.

For Swarm environments, PostgreSQL is internal-only. Connect through a container on the `backend` network or use an SSH tunnel if direct database access is needed.

Database Data Is Missing

Check whether the expected Docker volume exists:

```
docker volume ls
```

Production data should be in:

```
pg_data
```

Prometheus

Prometheus is the metrics collection and time-series storage service for the Lounastutka monitoring stack. Grafana uses Prometheus as the default metrics data source.

The project runs Prometheus in the Swarm environments:

- local Swarm testing with `compose.local.swarm.yaml`
- production Swarm deployment with `compose.yaml`

Prometheus is not part of the regular local development compose file.

Role In The System

Prometheus is responsible for:

- scraping configured metrics targets
- storing metric samples in its time-series database
- exposing a query API for Grafana
- providing the Prometheus web UI for direct metric inspection

Grafana is the main visualization UI, but Prometheus can also be opened directly for debugging targets and PromQL queries.

Service Configuration

Production uses the `prom/prometheus:latest` image:

```
prometheus:
  image: prom/prometheus:latest
  volumes:
    - ./docker/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prom_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--web.console.libraries=/usr/share/prometheus/console_libraries'
    - '--web.console.templates=/usr/share/prometheus/consoles'
```

The config file is stored in the repository at:

```
docker/prometheus/prometheus.yml
```

Inside the container it is mounted at:

```
/etc/prometheus/prometheus.yml
```

Routes

Production

In production, Prometheus is exposed through Traefik:

Route	Router	Internal port
`https://prometheus.swarm.lounastutka.fi`	`prometheus`	`9090`

The production route uses HTTPS, Let's Encrypt, and basic authentication:

```
- "traefik.http.routers.prometheus.rule=Host(`prometheus.swarm.lounastutka.fi`)"
- "traefik.http.routers.prometheus.entrypoints=websecure"
- "traefik.http.routers.prometheus.tls=true"
- "traefik.http.routers.prometheus.tls.certresolver=le"
- "traefik.http.routers.prometheus.middlewares=prometheus-auth@swarm"
- "traefik.http.services.prometheus.loadbalancer.server.port=9090"
```

The basic-auth middleware is configured in the production compose labels:

```
- "traefik.http.middlewares.prometheus-auth.basicauth.users=admin:$$apr1$$zcBUgvAR$$482EWeJ0
```

Local Swarm

Local Swarm exposes Prometheus through Traefik without TLS:

Route	Router	Internal port
`http://prometheus.localhost`	`prometheus`	`9090`

Scrape Configuration

The global scrape and evaluation interval is `15s`:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
```

The current active scrape target is Prometheus itself:

```
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
```

There is also a commented example for scraping the Bun backend application:

```
# - job_name: 'bun_app'
#   static_configs:
#     - targets: ['app:3001']
```

Enable application scraping only after the target service exposes Prometheus-format metrics.

Grafana Data Source

Grafana provisions Prometheus as the default data source from:

```
docker/grafana/provisioning/datasources/datasources.yaml
```

The configured data source URL is:

```
http://prometheus:9090
```

Grafana reaches Prometheus by service name on the Docker network.

Networks

Prometheus joins three Swarm networks:

Network	Purpose
`monitoring`	Connects Prometheus to Grafana and other monitoring services.
`backend`	Allows Prometheus to scrape backend services when metrics are enabled.
`traefik_proxy`	Allows Traefik to expose the Prometheus web UI.

Prometheus must stay connected to `traefik_proxy` for the public route and to `monitoring` for Grafana access.

Storage

Prometheus stores its time-series database at:

```
/prometheus
```

Production persists this directory in the named volume:

```
prom_data
```

Local Swarm uses:

```
prom_data_local_swarm
```

Removing the volume removes historical Prometheus metrics.

Placement

Prometheus runs as one replica on a Swarm manager node:

```yaml
deploy:
  replicas: 1
  placement:
    constraints:
      - node.role == manager
```

Relationship To Traefik Metrics

Traefik has Prometheus metrics enabled in the Traefik service configuration:

```
--metrics.prometheus=true
```

In production, Traefik serves metrics on its internal metrics entrypoint:

```
--metrics.prometheus.entrypoint=metrics
--entrypoints.metrics.address=:8082
```

The current Prometheus configuration does not yet scrape Traefik. Add a scrape job for the Traefik metrics endpoint when Traefik metrics should be stored in Prometheus.

Useful Commands

Show the Prometheus service:

```
docker service ps lounastutka_prometheus
```

Follow Prometheus logs:

```
docker service logs -f lounastutka_prometheus
```

Inspect the Prometheus service:

```
docker service inspect lounastutka_prometheus
```

List stack services:

```
docker stack services lounastutka
```

For local Swarm, deploy the stack:

```
docker stack deploy -c compose.local.swarm.yaml lounastutka
```

Then open:

```
http://prometheus.localhost
```

Troubleshooting

Prometheus Route Returns 404

A Traefik 404 usually means the request did not match a router.

Check that:

- the hostname is `prometheus.swarm.lounastutka.fi` in production
- the hostname is `prometheus.localhost` in local Swarm
- the Prometheus service has `traefik.enable=true`
- the router uses the correct entrypoint
- Swarm labels are under `deploy.labels`

Prometheus Route Returns Bad Gateway

A bad gateway means Traefik matched the route but could not reach Prometheus.

Check that:

- the Prometheus task is running
- Prometheus and Traefik both use the `traefik_proxy` network
- the load balancer port is `9090`
- Prometheus is listening on port `9090`

Grafana Cannot Query Prometheus

Check that:

- Grafana and Prometheus share the `monitoring` network
- the Grafana data source URL is `http://prometheus:9090`
- the Prometheus service is running
- Prometheus is listening on port `9090`

Useful commands:

```
docker service logs -f lounastutka_grafana
docker service logs -f lounastutka_prometheus
```

Target Is Down

Open the Prometheus targets page and check the target status:

```
https://prometheus.swarm.lounastutka.fi/targets
```

For local Swarm:

```
http://prometheus.localhost/targets
```

If a target is down, check:

- the target service is running
- the target service shares a network with Prometheus
- the target port is correct
- the target exposes Prometheus-format metrics

Metrics Disappear After Removing Volumes

Prometheus stores metrics in the `prom_data` volume. If the volume is removed, historical metrics are deleted.

Grafana

Grafana is the visualization layer for the Lounastutka monitoring stack. It is used to view metrics from Prometheus and logs from Loki.

The project runs Grafana in the Swarm environments:

- local Swarm testing with `compose.local.swarm.yaml`
- production Swarm deployment with `compose.yaml`

Grafana is not part of the regular local development compose file.

Role In The System

Grafana is responsible for:

- providing dashboards for application and infrastructure monitoring
- querying Prometheus for metrics
- querying Loki for logs
- giving one web UI for monitoring the deployed stack
- storing Grafana state in a named Docker volume

Grafana does not collect metrics or logs by itself. Prometheus scrapes metrics, Promtail ships logs to Loki, and Grafana reads from those services.

Service Configuration

Production uses the `grafana/grafana:latest` image:

```
grafana:
  image: grafana/grafana:latest
  expose:
    - "3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GF_ADMIN_PASSWORD}
  volumes:
    - grafana_data:/var/lib/grafana
    - ./docker/grafana/provisioning:/etc/grafana/provisioning:ro
```

The container listens internally on port `3000`. Traefik routes external traffic to that port.

Routes

Production

In production, Grafana is exposed through Traefik:

Route	Router	Internal port
`https://grafana.lounastutka.fi`	`grafana`	`3000`

The production route uses HTTPS and the Let's Encrypt certificate resolver named `le`:

```
- "traefik.http.routers.grafana.rule=Host(`grafana.lounastutka.fi`)"
- "traefik.http.routers.grafana.entrypoints=websecure"
- "traefik.http.routers.grafana.tls=true"
- "traefik.http.routers.grafana.tls.certresolver=le"
- "traefik.http.services.grafana.loadbalancer.server.port=3000"
```

Local Swarm

Local Swarm exposes Grafana through Traefik without TLS:

Route	Router	Internal port
`http://grafana.localhost`	`grafana`	`3000`

The local Swarm admin password defaults to `admin` if `GF_ADMIN_PASSWORD` is not set:

```
GF_SECURITY_ADMIN_PASSWORD: ${GF_ADMIN_PASSWORD:-admin}
```

Authentication

Grafana uses its built-in login screen.

The admin password is configured with:

```
GF_ADMIN_PASSWORD
```

The compose files pass this value to Grafana as:

```
GF_SECURITY_ADMIN_PASSWORD
```

Set `GF_ADMIN_PASSWORD` in the deployment environment before deploying production. The username is the Grafana default admin user unless it is changed in Grafana configuration.

Provisioning

Grafana provisioning files are mounted read-only from:

```
docker/grafana/provisioning
```

Inside the container they are available at:

```
/etc/grafana/provisioning
```

The current provisioning config adds two data sources from `docker/grafana/provisioning/datasources/datasources.yaml`.

Data source	Type	URL	Default
`Prometheus`	`prometheus`	`http://prometheus:9090`	Yes
`Loki`	`loki`	`http://loki:3100`	No

Both data sources use proxy access, so Grafana connects to them from inside the Docker network.

Networks

Grafana joins two Swarm networks:

Network	Purpose
`monitoring`	Connects Grafana to Prometheus and Loki.
`traefik_proxy`	Allows Traefik to route browser traffic to Grafana.

Prometheus and Loki are addressed by service name:

```
http://prometheus:9090
http://loki:3100
```

Because those names resolve inside the `monitoring` network, Grafana must stay attached to that network.

Storage

Grafana stores its database, plugins, and runtime state in:

```
/var/lib/grafana
```

Production persists that directory in the named volume:

```
grafana_data
```

Local Swarm uses:

```
grafana_data_local_swarm
```

Removing this volume removes local Grafana state, including dashboards created through the UI.

Dependencies

Grafana depends on:

```yaml
depends_on:
  - prometheus
  - loki
```

In Swarm mode, `depends_on` does not wait for the dependency services to be healthy. If Grafana starts before Prometheus or Loki is ready, the data sources may show connection errors briefly and recover after the services start.

Useful Commands

Show the Grafana service:

```
docker service ps lounastutka_grafana
```

Follow Grafana logs:

```
docker service logs -f lounastutka_grafana
```

Inspect the service:

```
docker service inspect lounastutka_grafana
```

List monitoring services:

```
docker stack services lounastutka
```

For local Swarm, deploy the stack and open Grafana:

```
docker stack deploy -c compose.local.swarm.yaml lounastutka
```

Then visit:

```
http://grafana.localhost
```

Troubleshooting

Grafana Route Returns 404

A Traefik 404 usually means the request did not match a router.

Check that:

- the hostname is `grafana.lounastutka.fi` in production
- the hostname is `grafana.localhost` in local Swarm
- the Grafana service has `traefik.enable=true`
- the router uses the correct entrypoint
- Swarm labels are under `deploy.labels`

Grafana Route Returns Bad Gateway

A bad gateway means Traefik matched the route but could not reach Grafana.

Check that:

- the Grafana task is running

- Grafana and Traefik both use the `traefik_proxy` network
- the load balancer port is `3000`
- the Grafana container is listening on port `3000`

Data Source Cannot Connect

If Prometheus or Loki shows as unavailable in Grafana, check that:

- Grafana is connected to the `monitoring` network
- the `prometheus` service is running on port `9090`
- the `loki` service is running on port `3100`
- the provisioning URLs use service names, not public hostnames

Useful log commands:

```
docker service logs -f lounastutka_grafana
docker service logs -f lounastutka_prometheus
docker service logs -f lounastutka_loki
```

Admin Login Fails

Check the value of:

```
GF_ADMIN_PASSWORD
```

If the password was changed after Grafana already initialized, the persisted Grafana database in `grafana_data` may still contain the old password. Reset the password through Grafana administration tools or remove the Grafana volume only if losing stored Grafana state is acceptable.

Loki

Loki is the log storage service for the Lounastutka monitoring stack. Promtail collects Docker container logs and pushes them to Loki, and Grafana queries Loki to show logs in the Grafana UI.

The project runs Loki in the Swarm environments:

- local Swarm testing with `compose.local.swarm.yaml`
- production Swarm deployment with `compose.yaml`

Loki is not part of the regular local development compose file.

Role In The System

Loki is responsible for:

- receiving log streams from Promtail
- storing container logs
- exposing a query API for Grafana
- keeping logs internal to the monitoring network

Loki is not exposed through Traefik. Grafana is the browser-facing UI for reading logs.

Service Configuration

Production uses the `grafana/loki:latest` image:

```yaml
loki:
  image: grafana/loki:latest
  expose:
    - "3100"
  command: -config.file=/etc/loki/local-config.yaml
  networks:
    - monitoring
```

The container listens internally on port `3100`.

The service uses Loki's built-in local configuration:

```
/etc/loki/local-config.yaml
```

There is no project-specific Loki config file mounted at the moment.

Network Access

Loki is connected only to the `monitoring` network.

Service	Direction	URL
Promtail	Pushes logs to Loki	`http://loki:3100/loki/api/v1/push`
Grafana	Queries logs from Loki	`http://loki:3100`

Because Loki is addressed by service name, Promtail and Grafana must also be connected to the `monitoring` network.

Routes

Loki has no public route.

Environment	Public URL
Production	None
Local Swarm	None

To read logs, open Grafana instead:

Environment	Grafana URL
Production	`https://grafana.lounastutka.fi`
Local Swarm	`http://grafana.localhost`

Log Flow

The logging pipeline is:

```
Docker containers -> Promtail -> Loki -> Grafana
```

Promtail discovers Docker containers through the Docker socket:

```yaml
docker_sd_configs:
  - host: unix:///var/run/docker.sock
    refresh_interval: 5s
```

Promtail sends logs to Loki with:

```yaml
clients:
  - url: http://loki:3100/loki/api/v1/push
```

Grafana provisions Loki as a data source with:

```yaml
- name: Loki
  type: loki
  access: proxy
  url: http://loki:3100
```

Labels

Promtail currently adds these labels to log streams:

Label	Source
`container`	Docker container name
`streams`	Docker log stream

The labels are configured in `docker/promtail/config.yaml`:

```yaml
relabel_configs:
  - source_labels: [__meta_docker_container_name]
    target_label: container
  - source_labels: [__meta_docker_container_log_stream]
    target_label: streams
```

These labels can be used in Grafana Explore when querying Loki.

Example query:

```
{container=~".*app.*"}
```

Placement

Loki runs as one replica on a Swarm manager node:

```yaml
deploy:
  replicas: 1
  placement:
    constraints:
      - node.role == manager
```

Promtail runs globally, so every Swarm node can ship logs to the single Loki service.

Storage

The current compose files do not mount a named volume for Loki.

This means Loki uses the storage paths defined by the image's local configuration inside the container filesystem. If the Loki task is replaced, stored logs may be lost.

For longer retention or production durability, add an explicit Loki configuration file and mount persistent storage for Loki data.

Useful Commands

Show the Loki service:

```
docker service ps lounastutka_loki
```

Follow Loki logs:

```
docker service logs -f lounastutka_loki
```

Inspect the Loki service:

```
docker service inspect lounastutka_loki
```

Show Promtail logs:

```
docker service logs -f lounastutka_promtail
```

List stack services:

```
docker stack services lounastutka
```

For local Swarm, deploy the stack:

```
docker stack deploy -c compose.local.swarm.yaml lounastutka
```

Then use Grafana Explore with the Loki data source:

```
http://grafana.localhost
```

Troubleshooting

No Logs In Grafana

Check that:

- the Loki service is running
- the Promtail service is running on each node
- Grafana has the Loki data source provisioned
- Grafana and Loki share the `monitoring` network
- Promtail and Loki share the `monitoring` network

Useful commands:

```
docker service logs -f lounastutka_loki
docker service logs -f lounastutka_promtail
docker service logs -f lounastutka_grafana
```

Promtail Cannot Push Logs

Check that Promtail is using the internal Loki URL:

```
http://loki:3100/loki/api/v1/push
```

Also check that:

- the `loki` service name resolves on the `monitoring` network
- Loki is listening on port `3100`
- Promtail has the `monitoring` network attached

Grafana Data Source Fails

Check the provisioned Loki data source:

```
http://loki:3100
```

The URL should be the Docker service name, not a public hostname.

Loki Logs Disappear After Restart

The current Loki service does not use a named Docker volume. If a task is recreated, logs stored in the container filesystem can disappear.

Add persistent Loki storage before relying on Loki for long-term log retention.

Promtail

Promtail is the log collector for the Lounastutka monitoring stack. It discovers Docker containers, reads their logs, and pushes the log streams to Loki.

The project runs Promtail in the Swarm environments:

- local Swarm testing with `compose.local.swarm.yaml`
- production Swarm deployment with `compose.yaml`

Promtail is not part of the regular local development compose file.

Role In The System

Promtail is responsible for:

- discovering Docker containers through the Docker socket
- reading Docker container log files
- adding labels to log streams
- tracking read positions
- pushing logs to Loki

Promtail does not provide the UI for logs. Logs are viewed in Grafana through the Loki data source.

Service Configuration

Production uses the `grafana/promtail:latest` image:

```yaml
promtail:
  image: grafana/promtail:latest
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - /var/lib/docker/containers:/var/lib/docker/containers:ro
    - ./docker/promtail/config.yaml:/etc/promtail/config.yaml:ro
  command: -config.file=/etc/promtail/config.yaml
  networks:
    - monitoring
  deploy:
    mode: global
```

The config file is stored in the repository at:

```
docker/promtail/config.yaml
```

Inside the container it is mounted at:

```
/etc/promtail/config.yaml
```

Deployment Mode

Promtail runs in global mode:

```yaml
deploy:
  mode: global
```

This starts one Promtail task on every Swarm node. That matters because Docker container logs are stored on the node where each container runs.

Volumes

Promtail uses these mounts:

Host path	Container path	Purpose
`/var/run/docker.sock`	`/var/run/docker.sock`	Discover Docker containers.
`/var/lib/docker/containers`	`/var/lib/docker/containers`	Read Docker container log files.
`./docker/promtail/config.yaml`	`/etc/promtail/config.yaml`	Load Promtail configuration.

All mounts are read-only.

Network Access

Promtail is connected to the `monitoring` network.

It pushes logs to Loki with the internal service URL:

```
http://loki:3100/loki/api/v1/push
```

Loki must be reachable by service name on the same network.

Promtail HTTP Server

Promtail exposes its own HTTP server on port `9080` inside the container:

```yaml
server:
  http_listen_port: 9080
```

This port is not exposed through Traefik and is not published on the host. It is only for Promtail's own internal HTTP endpoints.

Positions File

Promtail tracks how far it has read in each log file with:

```yaml
positions:
  filename: /tmp/positions.yaml
```

The positions file is stored inside the Promtail container. Because it is not mounted to a persistent volume, a recreated Promtail task may reread some logs.

Docker Discovery

Promtail discovers containers using Docker service discovery:

```yaml
scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 5s
```

The refresh interval is `5s`, so Promtail notices new containers quickly.

Labels

Promtail adds labels from Docker metadata:

Label	Source
`container`	Docker container name
`streams`	Docker log stream

The labels are configured with:

```
relabel_configs:
  - source_labels: [__meta_docker_container_name]
    target_label: container
  - source_labels: [__meta_docker_container_log_stream]
    target_label: streams
```

These labels can be used in Grafana Explore with the Loki data source.

Example LogQL queries:

```
{container=~".*app.*"}
{streams="stdout"}
```

Log Flow

The logging pipeline is:

```
Docker containers -> Promtail -> Loki -> Grafana
```

Promtail pushes logs to Loki. Grafana queries Loki. Traefik is not involved in this internal log flow.

Useful Commands

Show Promtail tasks:

```
docker service ps lounastutka_promtail
```

Follow Promtail logs:

```
docker service logs -f lounastutka_promtail
```

Inspect the Promtail service:

```
docker service inspect lounastutka_promtail
```

Show Loki logs:

```
docker service logs -f lounastutka_loki
```

List stack services:

```
docker stack services lounastutka
```

For local Swarm, deploy the stack:

```
docker stack deploy -c compose.local.swarm.yaml lounastutka
```

Then use Grafana Explore with the Loki data source:

```
http://grafana.localhost
```

Troubleshooting

No Logs In Grafana

Check that:

- Promtail is running on each Swarm node
- Loki is running
- Grafana has the Loki data source provisioned
- Promtail and Loki share the `monitoring` network
- Grafana and Loki share the `monitoring` network

Useful commands:

```
docker service logs -f lounastutka_promtail
docker service logs -f lounastutka_loki
docker service logs -f lounastutka_grafana
```

Promtail Cannot Read Docker Logs

Check that the required mounts exist:

```
/var/run/docker.sock
/var/lib/docker/containers
```

Also check that both mounts are available on each Swarm node where Promtail runs.

Promtail Cannot Push To Loki

Check that the client URL is:

```
http://loki:3100/loki/api/v1/push
```

Then check that:

- the Loki service is running
- the `loki` service name resolves on the `monitoring` network
- Loki is listening on port `3100`
- Promtail is attached to the `monitoring` network

Duplicate Logs After Restart

The positions file is currently stored at:

```
/tmp/positions.yaml
```

Because this path is inside the Promtail container and is not backed by a volume, a recreated task can reread logs it has already shipped. Mount the positions file to persistent storage if duplicate logs after restarts become a problem.

Portainer

Portainer is the management UI for the Lounastutka Docker Swarm environment. It provides a browser interface for inspecting services, tasks, containers, networks, volumes, and node state.

The project runs Portainer in the Swarm environments:

- local Swarm testing with `compose.local.swarm.yaml`
- production Swarm deployment with `compose.yaml`

Portainer is not part of the regular local development compose file.

Role In The System

Portainer is responsible for:

- providing a web UI for Docker Swarm management
- connecting to Portainer agents running on Swarm nodes
- showing services, containers, logs, networks, and volumes
- storing Portainer configuration and UI state in a named Docker volume

Portainer is an operations tool. It is separate from the application runtime and from the monitoring stack.

Services

The Portainer setup has two services:

Service	Image	Purpose
`portainer_agent`	`portainer/agent:lts`	Runs on every node and exposes node/container detail
`portainer`	`portainer/portainer-ce:lts`	Runs the Portainer Community Edition web UI.

Agent Configuration

The agent runs in global mode:

```
portainer_agent:
  image: portainer/agent:lts
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - /var/lib/docker/volumes:/var/lib/docker/volumes
    - /:/host
  networks:
    - edge
  deploy:
    mode: global
    placement:
      constraints:
        - node.platform.os == linux
```

Global mode starts one agent task on every Linux Swarm node.

The agent mounts:

Host path	Container path	Purpose
`/var/run/docker.sock`	`/var/run/docker.sock`	Access Docker engine metadata.
`/var/lib/docker/volumes`	`/var/lib/docker/volumes`	Inspect Docker volumes.
`/`	`/host`	Inspect host filesystem data needed by Por

These mounts give Portainer broad operational visibility into the nodes. Keep the Portainer route and login protected.

Portainer UI Configuration

The Portainer UI service runs one replica:

```
portainer:
  image: portainer/portainer-ce:lts
  command: -H tcp://tasks.portainer_agent:9001 --tlsskipverify
  volumes:
    - portainer_data:/data
  networks:
    - edge
    - traefik_proxy
```

Portainer connects to all agent tasks with:

```
tcp://tasks.portainer_agent:9001
```

The `tasks.portainer_agent` DNS name resolves to the agent tasks in the Swarm service.

The UI listens internally on port `9000`. Traefik routes external browser traffic to that port.

Routes

Production

In production, Portainer is exposed through Traefik:

Route	Router	Internal port
`https://portainer.lounastutka.fi`	`portainer`	`9000`

The production route uses HTTPS and the Let's Encrypt certificate resolver named `le`:

```
- "traefik.http.routers.portainer.rule=Host(`portainer.lounastutka.fi`)"
- "traefik.http.routers.portainer.entrypoints=websecure"
- "traefik.http.routers.portainer.tls=true"
- "traefik.http.routers.portainer.tls.certresolver=le"
- "traefik.http.services.portainer.loadbalancer.server.port=9000"
```

Local Swarm

Local Swarm exposes Portainer through Traefik without TLS:

Route	Router	Internal port
`http://portainer.localhost`	`portainer`	`9000`

Networks

Portainer uses two Swarm networks:

Network	Used by	Purpose
`edge`	`portainer`, `portainer_agent`	Internal Portainer-to-agent communication.

Network	Used by	Purpose
`traefik_proxy`	`portainer`, `traefik`	Public route through Traefik.

The agent does not need to join `traefik_proxy` because browser traffic goes only to the Portainer UI service.

Storage

Portainer stores its application data at:

```
/data
```

Production persists that directory in:

```
portainer_data
```

Local Swarm uses:

```
portainer_data_local_swarm
```

Removing this volume removes Portainer state, including users, settings, and environment configuration.

Placement

The Portainer UI runs on a manager node:

```yaml
deploy:
  replicas: 1
  placement:
    constraints:
      - node.role == manager
```

The Portainer agent runs on every Linux node:

```yaml
deploy:
  mode: global
  placement:
    constraints:
      - node.platform.os == linux
```

Useful Commands

Show the Portainer UI service:

```
docker service ps lounastutka_portainer
```

Show the Portainer agent tasks:

```
docker service ps lounastutka_portainer_agent
```

Follow Portainer UI logs:

```
docker service logs -f lounastutka_portainer
```

Follow Portainer agent logs:

```
docker service logs -f lounastutka_portainer_agent
```

Inspect the Portainer service:

```
docker service inspect lounastutka_portainer
```

List stack services:

```
docker stack services lounastutka
```

For local Swarm, deploy the stack:

```
docker stack deploy -c compose.local.swarm.yaml lounastutka
```

Then open:

```
http://portainer.localhost
```

Troubleshooting

Portainer Route Returns 404

A Traefik 404 usually means the request did not match a router.

Check that:

- the hostname is `portainer.lounastutka.fi` in production
- the hostname is `portainer.localhost` in local Swarm
- the Portainer service has `traefik.enable=true`
- the router uses the correct entrypoint
- Swarm labels are under `deploy.labels`

Portainer Route Returns Bad Gateway

A bad gateway means Traefik matched the route but could not reach Portainer.

Check that:

- the Portainer task is running
- Portainer and Traefik both use the `traefik_proxy` network
- the load balancer port is `9000`
- the Portainer container is listening on port `9000`

Portainer Cannot Connect To The Agent

Check that:

- `portainer_agent` has one task on each Linux node
- `portainer` and `portainer_agent` share the `edge` network
- the Portainer command points to `tcp://tasks.portainer_agent:9001`
- the agent service is healthy

Useful commands:

```
docker service ps lounastutka_portainer_agent
docker service logs -f lounastutka_portainer_agent
docker service logs -f lounastutka_portainer
```

Portainer State Is Missing

Portainer stores state in the `portainer_data` volume. If the volume is removed or replaced, Portainer may start as a fresh installation.

Check the stack volumes:

```
docker volume ls
```

Login Fails

Portainer manages its own users inside the data volume. If login fails, check whether the `portainer_data` volume is the expected one and whether the service was redeployed with a fresh volume.